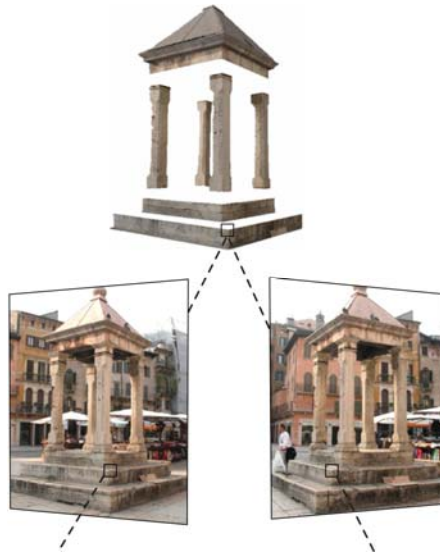


Two-view geometry

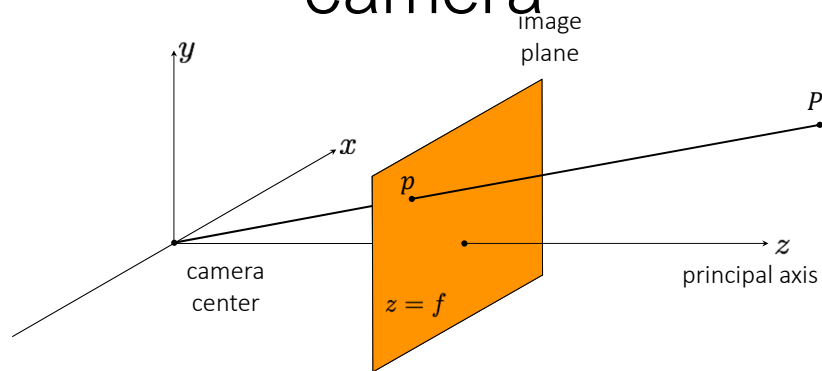


Slide credits: Ioannis Gkioulekas, Kris Kitani, Fredo Durand

Overview of today's lecture

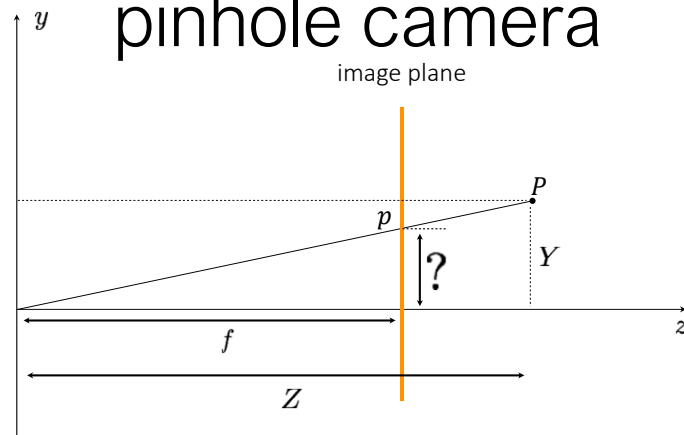
- Leftover from previous lecture: camera calibration.
- Triangulation.
- Epipolar geometry.
- Essential matrix.
- Fundamental matrix.
- 8-point algorithm.

The (rearranged) pinhole camera



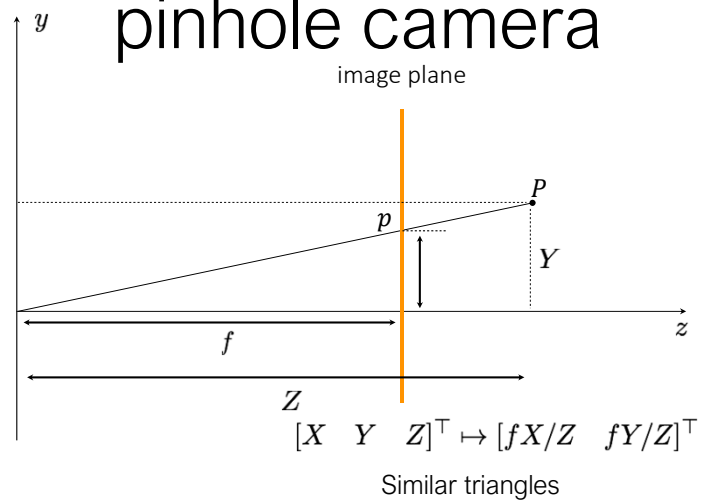
What is the equation for image coordinate $\mathbf{p}$ in terms of $\mathbf{P}$?

The 2D view of the (rearranged) pinhole camera



What is the equation for image coordinate $\mathbf{p}$ in terms of $\mathbf{P}$?

The 2D view of the (rearranged) pinhole camera



The pinhole camera matrix

Relationship from similar triangles:

$$\begin{bmatrix} X & Y & Z \end{bmatrix}^T \mapsto \begin{bmatrix} fX/Z & fY/Z \end{bmatrix}^T$$

General camera model:

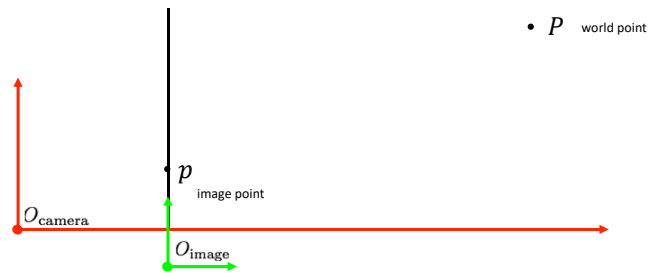
$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

What does the pinhole camera projection look like?

$$\mathbf{M} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

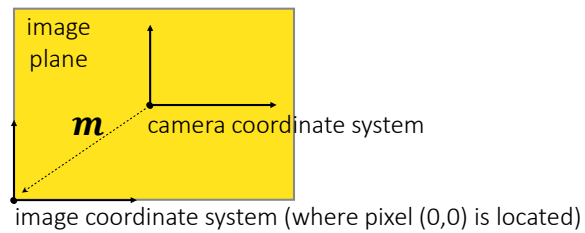
Generalizing the camera matrix

In general, the camera and image have *different* coordinate systems.



Generalizing the camera matrix

In particular, the camera origin and image origin may be different:



How does the camera matrix change?

$$\mathbf{M} = \begin{bmatrix} f & 0 & m_x & 0 \\ 0 & f & m_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

← shift vector
transforming
camera origin to
image origin

Camera matrix decomposition

We can decompose the camera matrix like this:

$$M = \begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & | & 0 \\ 0 & 1 & 0 & | & 0 \\ 0 & 0 & 1 & | & 0 \end{bmatrix}$$

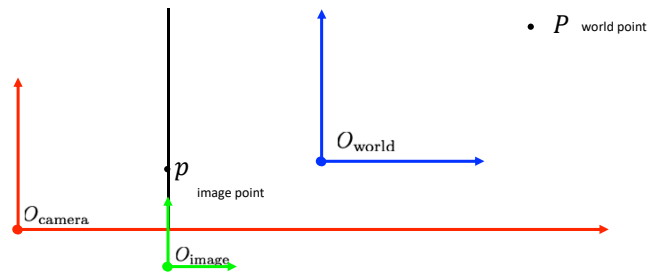
(homogeneous) transformation
from 2D to 2D, accounting for not
unit focal length and origin shift

(homogeneous) projection from 3D
to 2D, assuming image plane at $f = 1$
and shared camera/image origin

Also written as: $M = K[I|0]$ where $K = \begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix}$

Generalizing the camera matrix

In general, there are *three*, generally different, coordinate systems.



We need to know the transformations between them.

General pinhole camera matrix

$$\mathbf{M} = \mathbf{K} [\mathbf{R}|\mathbf{t}]$$

$$\mathbf{M} = \underbrace{\begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix}}_{\substack{\text{intrinsic} \\ \text{parameters}}} \underbrace{\begin{bmatrix} r_1 & r_2 & r_3 & | & t_1 \\ r_4 & r_5 & r_6 & | & t_2 \\ r_7 & r_8 & r_9 & | & t_3 \end{bmatrix}}_{\substack{\text{extrinsic} \\ \text{parameters}}}$$

$$\mathbf{R} = \begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ r_7 & r_8 & r_9 \end{bmatrix} \quad \mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ t_3 \end{bmatrix}$$

3D rotation 3D translation

General pinhole camera matrix

$$\mathbf{M} = \mathbf{K}\mathbf{R}[\mathbf{I} | -\mathbf{C}] \quad (\text{translate first then rotate})$$

$$\mathbf{M} = \mathbf{K} [\mathbf{R} | \mathbf{t}] \quad (\text{rotate first then translate})$$

$$\text{where } \mathbf{t} = -\mathbf{R}\mathbf{C}$$

Q: What is the projection of the camera center $\mathbf{C}$?

$$\text{A: } \mathbf{M}\mathbf{C} = \mathbf{0}$$

	Structure (scene geometry)	Motion (camera geometry)	Measurements
Camera Calibration (a.k.a. Pose Estimation)	known	estimate	3D to 2D correspondences
Triangulation	estimate	known	2D to 2D corespondences
Reconstruction	estimate	estimate	2D to 2D correspondences

Our ultimate goal in this class is to do 3D reconstruction. We will see that in order to do that we need to detect the projection of the same 3D points in multiple images taken from different viewpoints. We will be referring to these as image captured by multiple cameras, though it can also be the same camera moved to a different position. We will discuss next why one camera is not enough.

For that we want to start with a set of corresponding 2D points in a set of images and recover their 3D position. By corresponding points I mean that we know to detect the projection of the same 3D point in more than one image.

In order to solve this problem we need to solve a few sub problems. The first one is camera calibration that we have already discussed. In camera calibration we assume that we know the 3D position of a point and the corresponding projection in a 2D image. From a few pairs of matching points we want to recover the camera matrix.

The next estimation problem we are going to discuss is triangulation. In triangulation we assume two (or more) camera matrices are known and we are

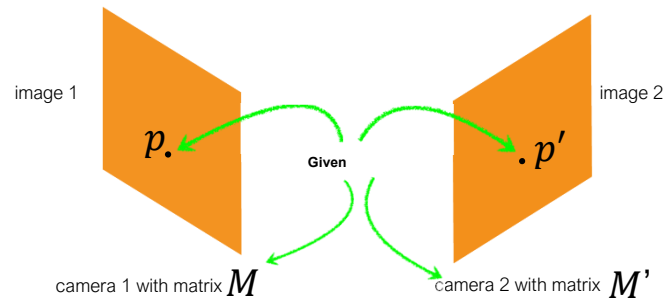
also given corresponding 2D points in the images. From this we want to recover the 3D position.

Finally we are going to discuss the reconstruction problem, where we are only given a set of corresponding 2D points and we want to recover both the camera matrices and the 3D positions.

It is common to address these tasks as “structure” and “motion”, where structure refers to recover the 3D position, and motion refers to estimating the camera matrices. As mentioned the camera matrices are often the same camera moved to a different viewpoint and therefore by recovering the camera matrices we are essentially recovering the motion of the photographer.

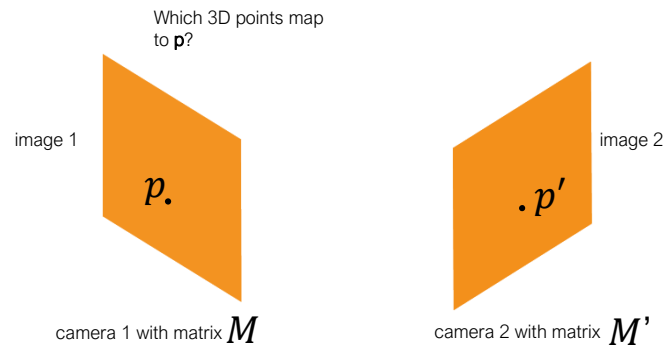
Triangulation

Triangulation



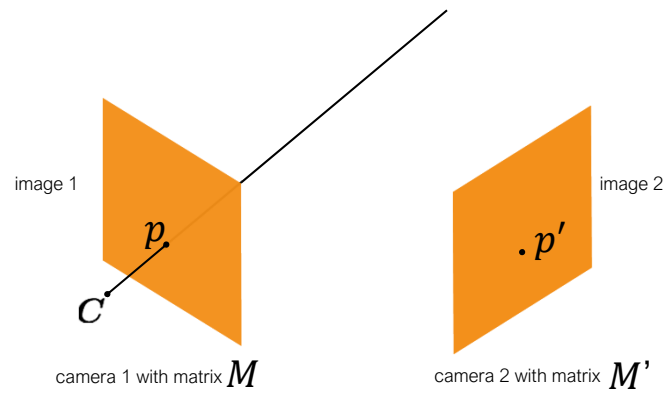
We will start with the following setting: We have two images of the same scene, captured by *calibrated* cameras from two different viewpoints, were by calibrated cameras, I mean that the camera matrices M are known. We identify two pixels, one in each image, that both look at the same point in the scene. Then, we want to reconstruct the 3D location of this point.

Triangulation



A first thing to discuss here is why we cannot reconstruct the 3D point corresponding to the two image points from just one image. From what we discussed in the Camera Models lecture, we know that a camera maps **rays** to image points. In other words, given an image location p and the camera center C , we can cast the ray passing through the two points: Every 3D point on this ray maps to the same image point p , and we cannot tell which point on the ray is the real 3D point in the scene just from one image.

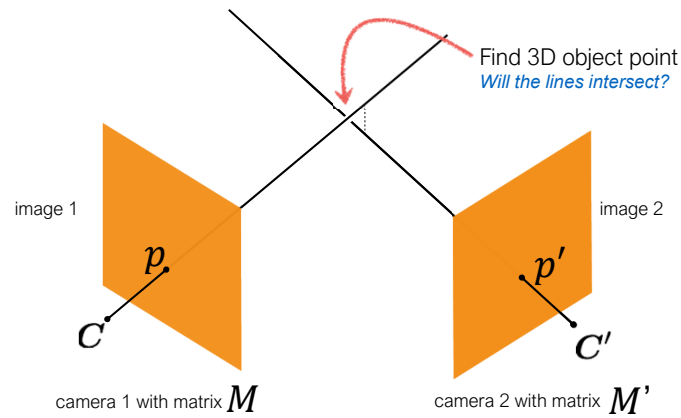
Triangulation



The 3D point lies somewhere along this ray.

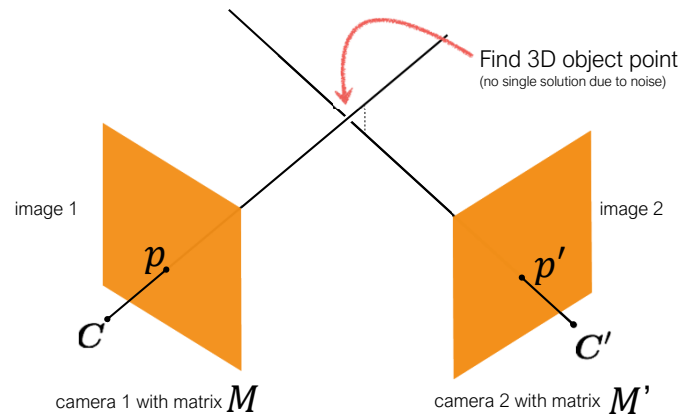
This is a ray starting from the camera center C (which we can compute as the nullspace of M) and going through the point p on the image plane

Triangulation



We can also look at the ray connecting p' and the second camera center C' . Remember that we assumed that image points p and p' correspond to the same 3D point. Therefore, we can now localize this point by finding where the two rays intersect! In practice, because of small errors in pixel localization and camera calibration, the two rays will not intersect exactly. So, we will perform their intersection in the least-squares sense, by finding the point that minimizes the sum of its squared distances from the two rays. We will formulate this least squares problem below.

Triangulation



Triangulation

Given a pair of (noisy) matched points

$$\{p, p'\}$$

and camera matrices

$$M, M'$$

Estimate the 3D point P

Such that

$$p = MP, \quad p' = M'P$$

Here is our problem setting then: We have the image coordinates for the two matched points, and the corresponding camera matrices, and we want to estimate from them the 3D point upper-case P. We will follow our usual procedure of setting up a linear system for estimating the 3D point.

$$p = MP$$

(homogeneous coordinate)

Also, this is a similarity relation because it involves homogeneous coordinates

$$p = \alpha MP$$

(homogeneous coordinate)

Same ray direction but differs by a scale factor

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

How do we solve for unknowns in a similarity relation?

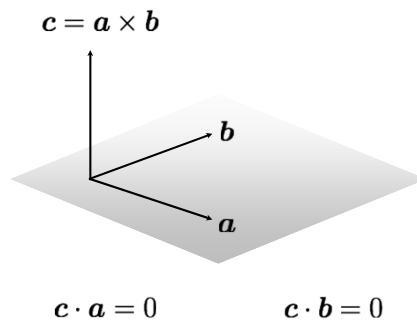
Remove scale factor, convert to linear system and solve with SVD!

Let's start by writing out the 3D to 2D mapping implemented from camera 1. Note that, to write this expression, we need to convert our input 2D points from heterogeneous to homogeneous coordinates. This means that, in addition to the unknown P, our mapping relationship will include some scalar a. This is because we do not know what value will be produced for the third coordinate of the 2D point by the camera matrix. The scalar a corresponds to dividing out the unknown scaling of the third coordinate in the homogeneous vector of the 2D point.

Recall: Cross Product

Vector (cross) product

takes two vectors and returns a vector perpendicular to both



$$a \times b = \begin{bmatrix} a_2b_3 - a_3b_2 \\ a_3b_1 - a_1b_3 \\ a_1b_2 - a_2b_1 \end{bmatrix}$$

cross product of two vectors in the same direction?

$$a \times a = 0$$

remember this!!!

To see how to deal with the unknown scalar a , we will use a linear algebra trick relating to the cross product between two vectors. Remember that the cross product takes two vectors and returns another vector that is perpendicular to both of them. We can write the cross product of two vectors analytically in terms of their coordinates. More importantly, we will use the following property of the cross product: If two vectors are parallel, then their cross product is zero.

$$\mathbf{p} = \alpha \mathbf{MP}$$

Same direction but differs by a scale factor

$$\mathbf{p} \times \mathbf{MP} = \mathbf{0}$$

Cross product of two vectors of same direction is zero
(this equality removes the scale factor)

Going back to triangulation, the relationship we wrote earlier means that the vectors $\mathbf{p}$, and $\mathbf{MP}$ are parallel to each other, given that they are equal up to a scale factor.

Therefore, their cross product will be zero. We can use this equality to rewrite the projection relationship in a way that removes the scale factor α .

$$p = \alpha MP$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} P$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} - & m_1^T & - \\ - & m_2^T & - \\ - & m_3^T & - \end{bmatrix} P$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1^T P \\ m_2^T P \\ m_3^T P \end{bmatrix}$$

Here is the same expression where we have written out every vector and matrix.
As we have done during the Camere Models lecture, we will represent the camera matrix in terms of 3 4x1 vectors equal to the transpose of its rows.

$$p = \alpha MP$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} P$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} - & m_1^T & - \\ - & m_2^T & - \\ - & m_3^T & - \end{bmatrix} P$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1^T P \\ m_2^T P \\ m_3^T P \end{bmatrix}$$

$$p \times MP = 0 \quad \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \times \begin{bmatrix} m_1^T P \\ m_2^T P \\ m_3^T P \end{bmatrix} = \begin{bmatrix} ym_3^T P - m_2^T P \\ m_1^T P - xm_3^T P \\ xm_2^T P - ym_1^T P \end{bmatrix} = 0$$

We now express the cross product constraint in terms of matrix elements

Expanding out the camera matrix expression, this gives us three linear relationships between the known 2D point and camera matrix, and the unknown 3D point.

Using the fact that the cross product should be zero

$$p \times MP = 0$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \times \begin{bmatrix} m_1^T P \\ m_2^T P \\ m_3^T P \end{bmatrix} = \begin{bmatrix} ym_3^T P - m_2^T P \\ m_1^T P - xm_3^T P \\ xm_2^T P - ym_1^T P \end{bmatrix} = 0$$

Third line is a linear combination of the first and second lines.
(x times the first line plus y times the second line)

One 2D to 3D point correspondence give you  equations

We note, however, that only two of these relationships are linearly independent. The third relationship is a linear combination of the first two. Mathematically, this means that one 2D point gives us two constraints on the three coordinates of the unknown 3D point. This is the mathematical expression of what we explained earlier, that we cannot reconstruct the 3D point just from one camera.

To proceed, we drop the third equation.

$$\begin{bmatrix} ym_3^T P - m_2^T P \\ m_1^T P - xm_3^T P \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} ym_3^T - m_2^T \\ m_1^T - xm_3^T \end{bmatrix} P = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$A_i P = 0$$

Now we can make a system of linear equations
(two lines for each 2D point correspondence)

Then we rearrange the remaining two equations to move the unknown 3D point P outside. We have now written all of the information we have from one camera into an *underdetermined* homogeneous linear system with the 3D point as the unknown.

Concatenate the 2D points from both images

$$\begin{bmatrix} ym_3^T - m_2^T \\ m_1^T - xm_3^T \\ y'm_3'^T - m_2'^T \\ m_1'^T - x'm_3'^T \end{bmatrix} P = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

sanity check! dimensions?

$$AP = 0$$

How do we solve homogeneous linear system?

SVD

To make this linear system well-posed, all we have to do is repeat the same process for camera 2, and stack its two relations together with those from camera 1.

This gives us the final homogeneous linear system, which we can solve using SVD to recover the unknown 3D point P.

	Structure (scene geometry)	Motion (camera geometry)	Measurements
Camera Calibration (a.k.a. Pose Estimation)	known	estimate	3D to 2D correspondences
Triangulation	estimate	known	2D to 2D coorespondences
Reconstruction	estimate	estimate	2D to 2D correspondences

We now move to our ultimate goal, of solving the structure and motion problem together. That is start from a set of 2D matching points in two or more images, and estimate both the camera matrices and the 3D positions.

This is a more challenging task and we will get to the actual answer only next week. Before that we need to learn about epipolar geometry.

Epipolar geometry

Remember what we need as input



Left image



Right image

Matching a point in the left image requires doing an expensive 2D search in the right image.

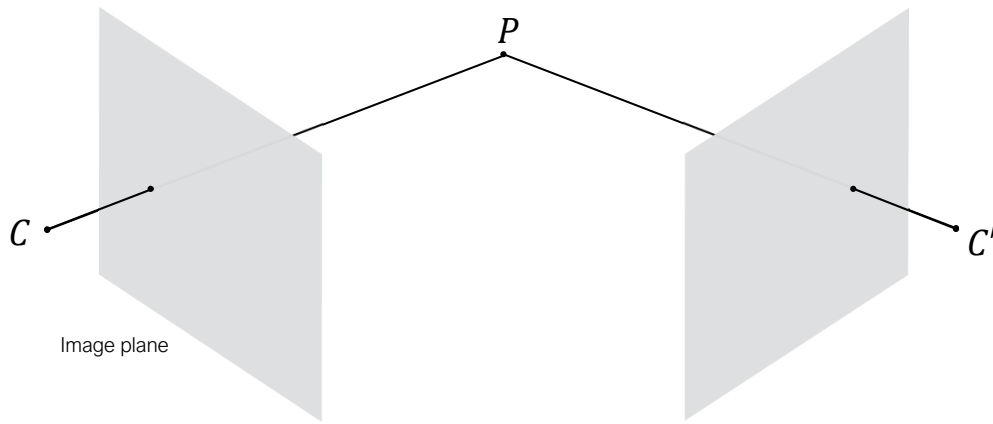
Going back to where we started, remember that to do all of this, we need to first have available a pair of matched points on the two camera images. Given a point on the left image, matching it would require performing a 2D search across all pixels of the right image, to find a pixel location that has the same appearance. Appearance here could be compared using the feature descriptors from the Feature Detection and Matching module. However, performing this 2D search can be impractically expensive. In the next slides, we will study how we can use geometry to simplify this search problem.

Disclaimer:

in the following slides we assume camera coordinate and image coordinate are the same

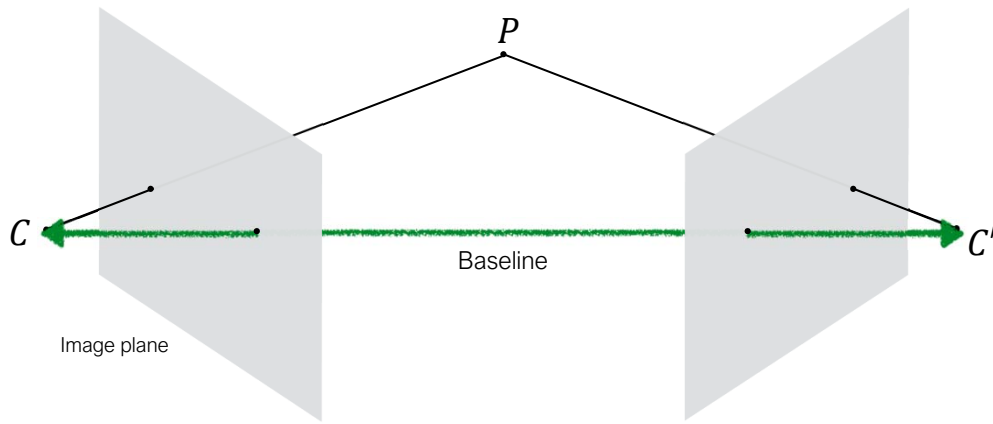
That is, internal matrix is $K=I$

Epipolar geometry



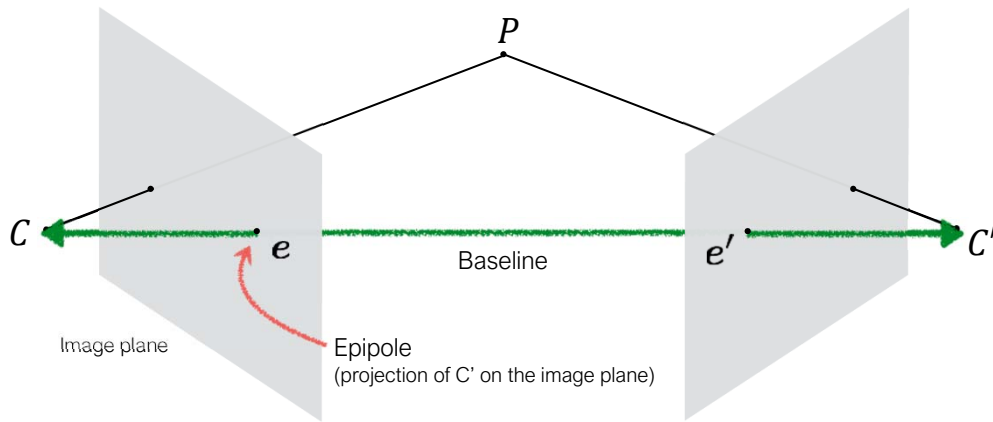
We will now go over a systematic way to describe the relative geometry of two cameras looking at the same scene, that goes by the name epipolar geometry. For now, we will be introducing a lot of terms and geometric concepts, before we see how we can use those to help with 3D reconstruction. As before, we assume we have two cameras, each with their camera center. We also draw the rays mapping a 3D point p to the image planes of the two cameras.

Epipolar geometry



We will refer to the line connecting the two camera centers as the baseline.

Epipolar geometry

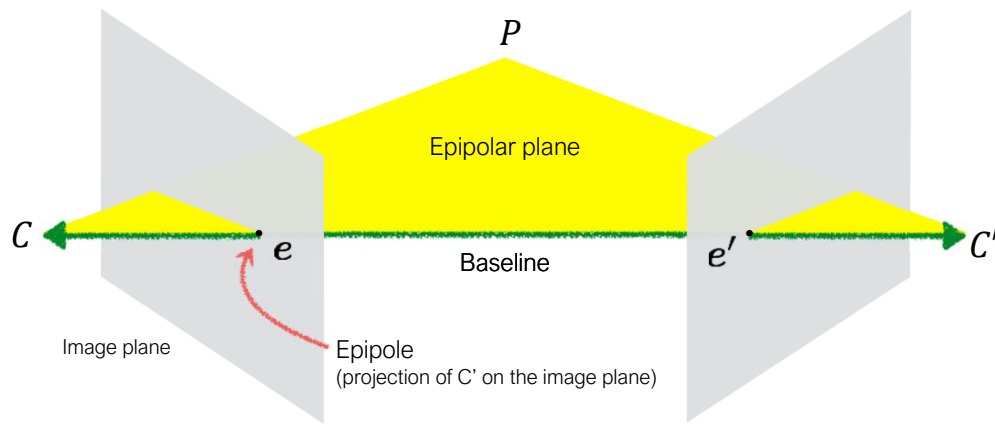


How do you compute the projection? $e = MC'$ $e' = M'C$

The baseline will intersect the image planes of the two cameras at two points that we will call the left and right epipole, for the left and right camera respectively.

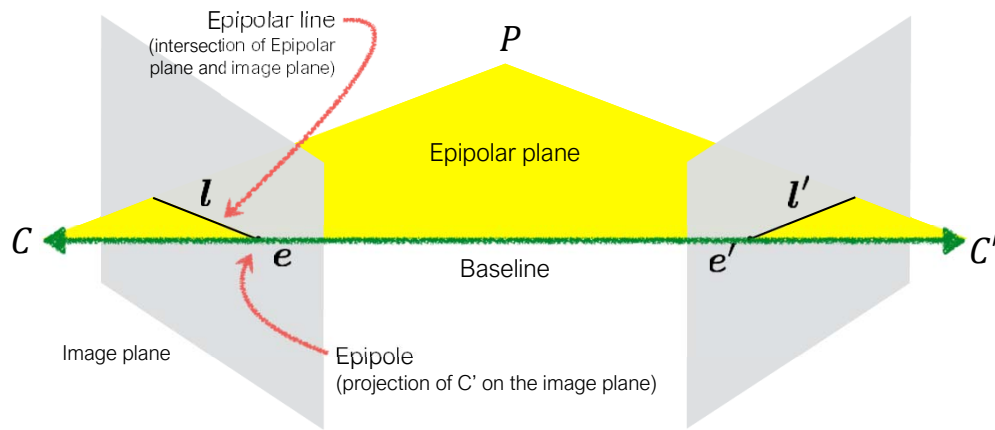
Recall that e is the projection of the second camera center C' on the first camera.
So we can compute it using MC'
And similarly we can compute e' .

Epipolar geometry



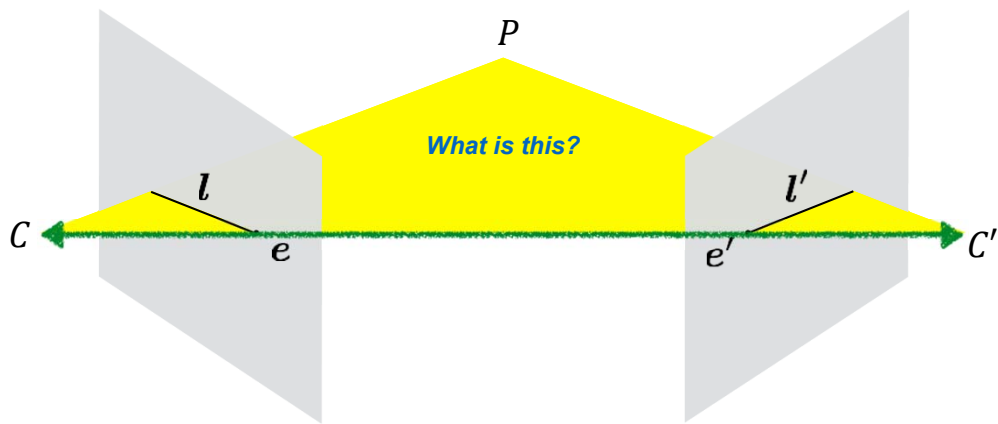
Now, suppose we are given some 3D point P . The point P and the baseline create a plane, that we will call the epipolar plane corresponding to P .

Epipolar geometry

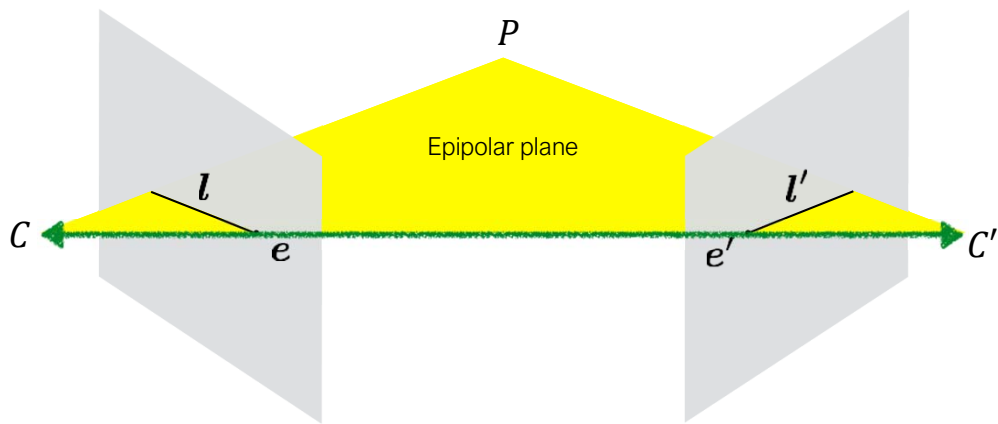


This plane intersects the two image planes at two lines, which we call the left and right epipolar line.

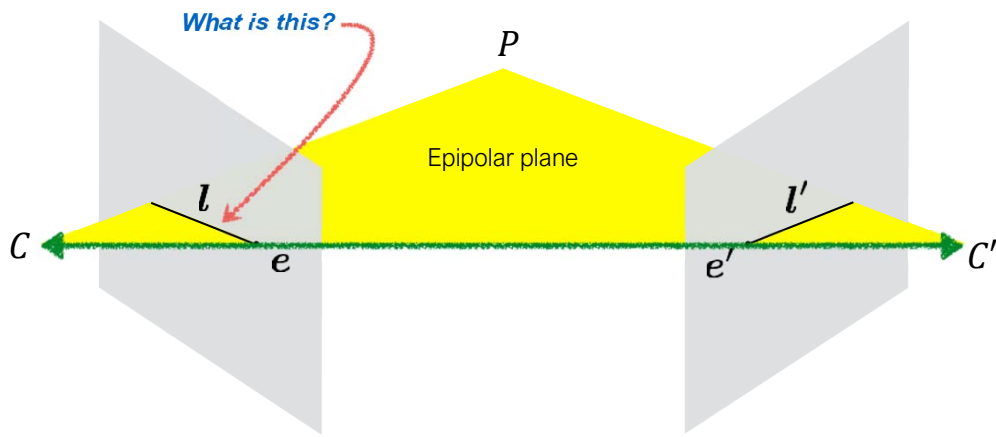
Quiz



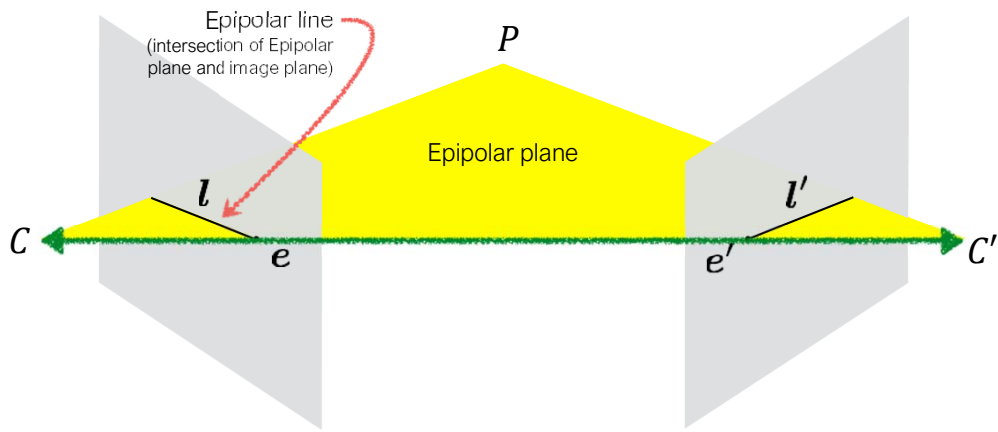
Quiz



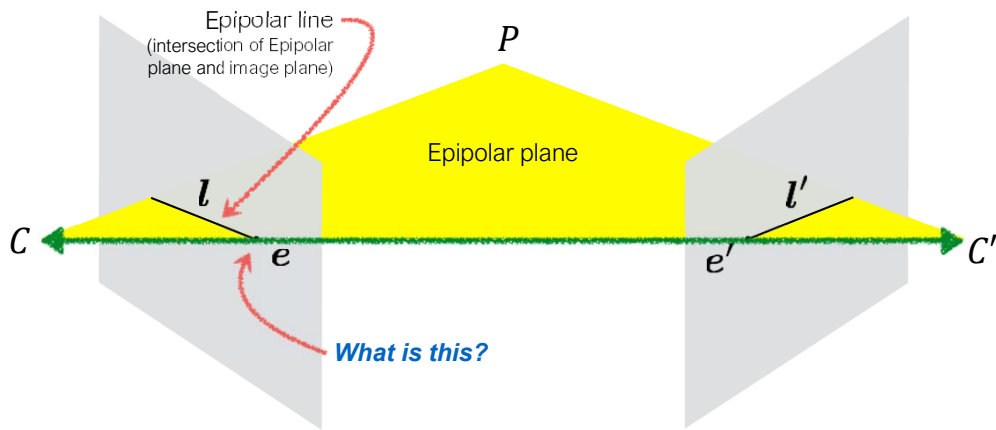
Quiz



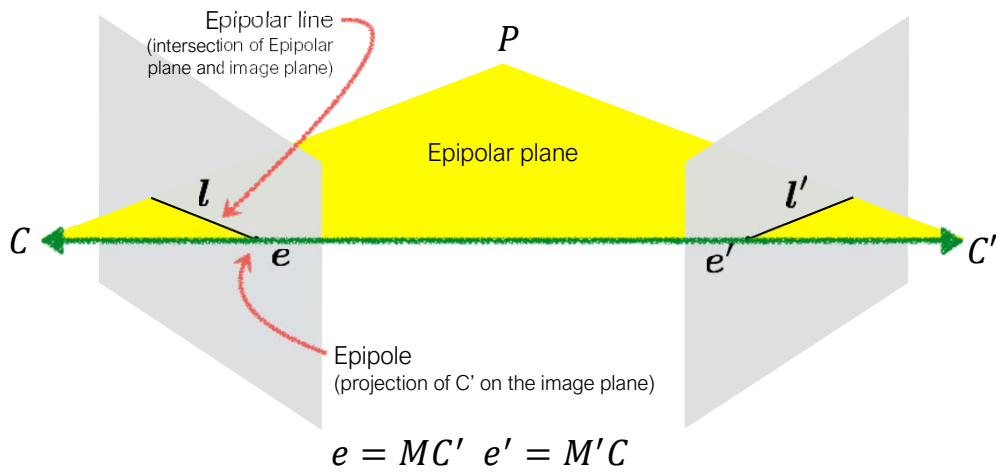
Quiz



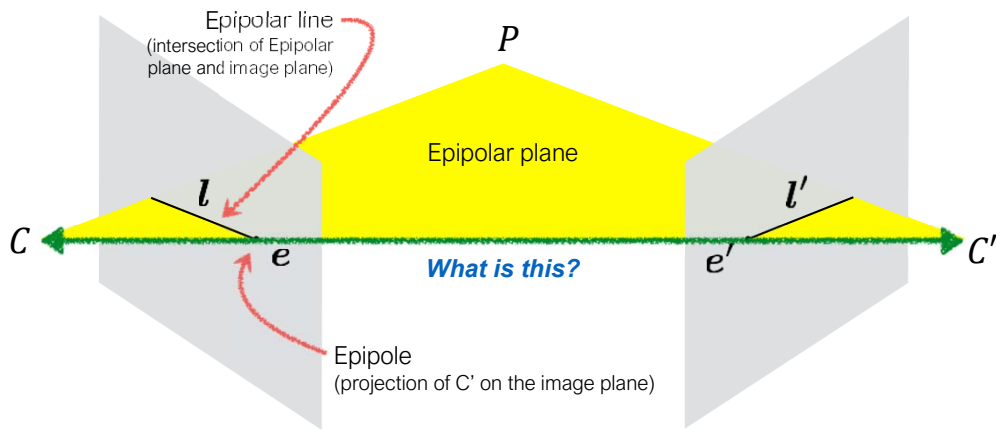
Quiz



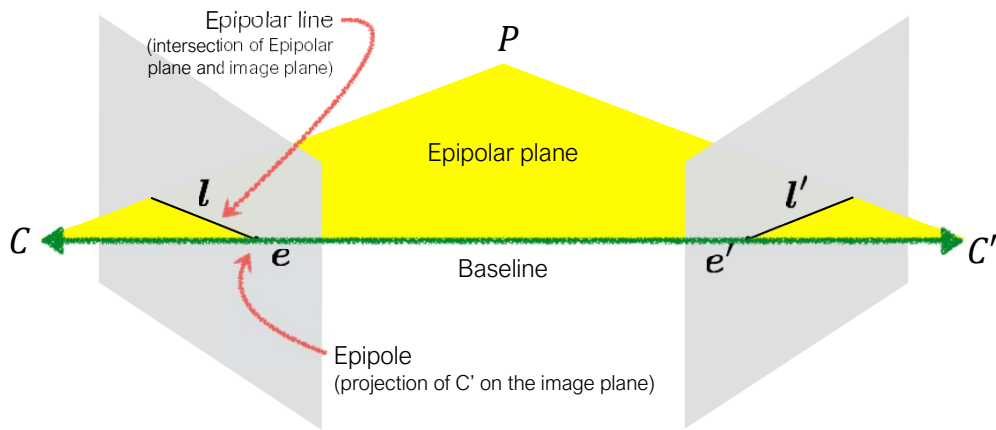
Quiz



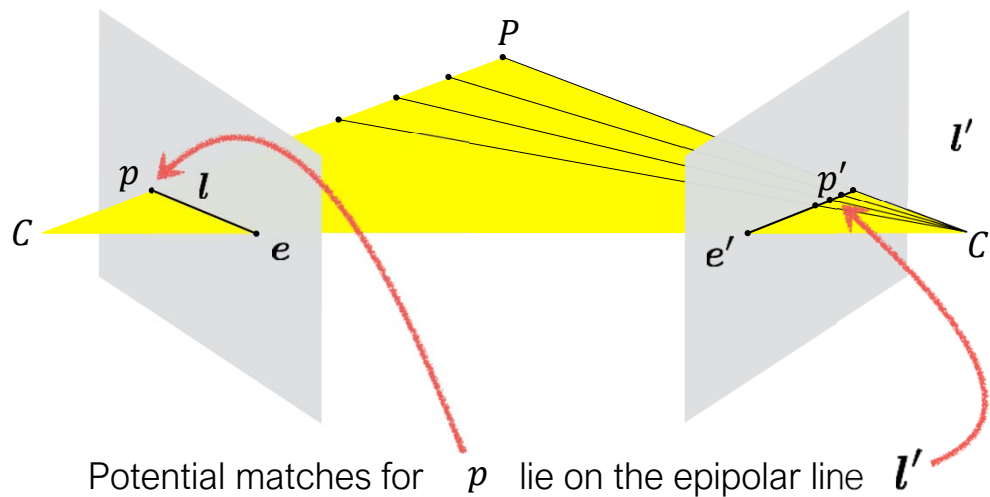
Quiz



Quiz



Epipolar constraint

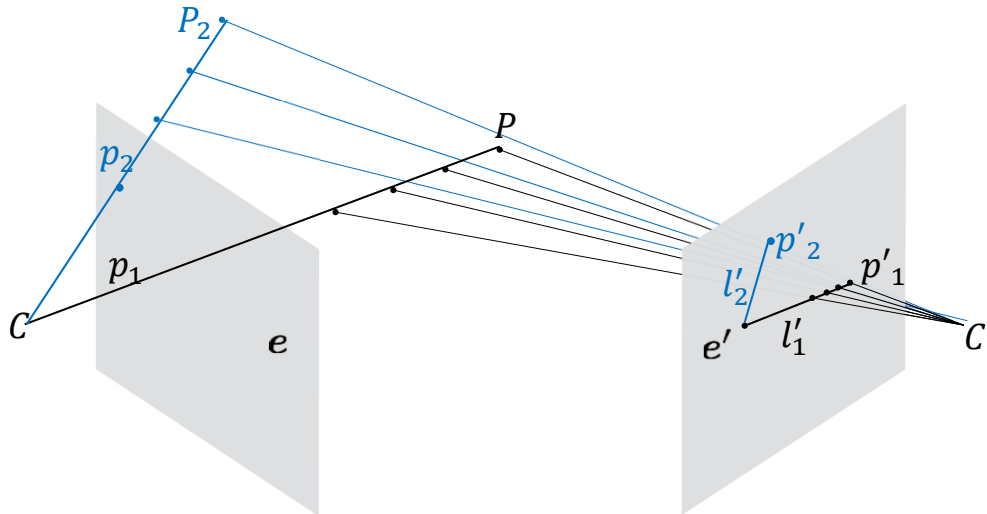


Now consider the 3D ray that starts from the camera center and passes through the image point p . This ray contains all of the 3D points in the scene that can map to image point p on the left camera. We can take each of these 3D points, and map it to the right camera. Doing so will trace on the image plane of the right camera the epipolar line l' we defined earlier. What this means is the following: Let's say that we have the image point p on the left image, and we want to search the right image for the matching point p' that looks at the same 3D point. This matching point can *only* be on the epipolar line l' , and cannot be anywhere else on the image.

We will see below that we can compute l' from the camera matrices.

This is a constraint that is available to us just by knowing the camera matrices and their relative pose, and completely independently of the scene. We will refer to this as the epipolar constraint.

Multiple epipolar lines



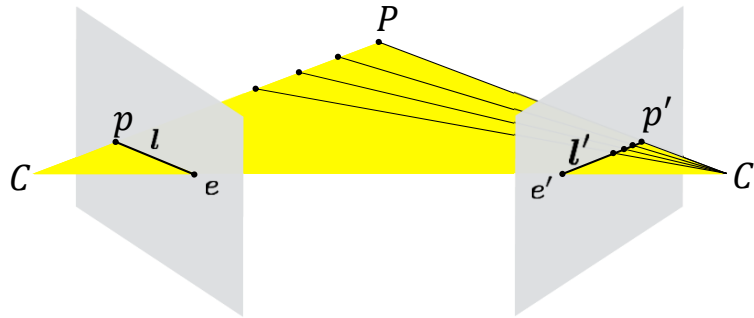
Different points p, p_2 in image 1 define different rays of potential 3D points that could project to them. Hence they also define different epipolar lines l'_1, l'_2 .

These lines intersect at the epipole e' , since all 3D rays intersect at C and e' is the projection of C to camera 2

For different points p on camera 1 we get different rays of 3D points that could be mapped to them.

Different rays define different epipolar lines.

For every p , the ray of 3D points it defines is passing through the camera center C ,
 Since e' is the projection of C on camera 2, e' is also a point on the epipolar line l' .
 So all the epipolar lines intersect at the epipole.



The point **p** (left image) maps to a Epipolar line in the right image

The baseline connects the Camera centers and epipols

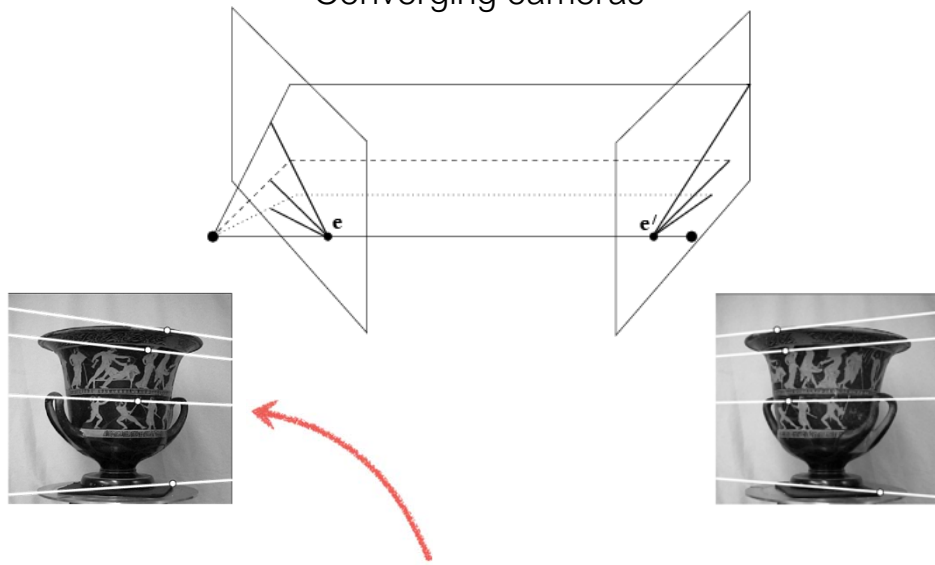
An epipole **e** is a projection of the Camera center C' on the image plane

All epipolar lines in an image intersect at the epipole

Here is a summary of properties we have defined so far.

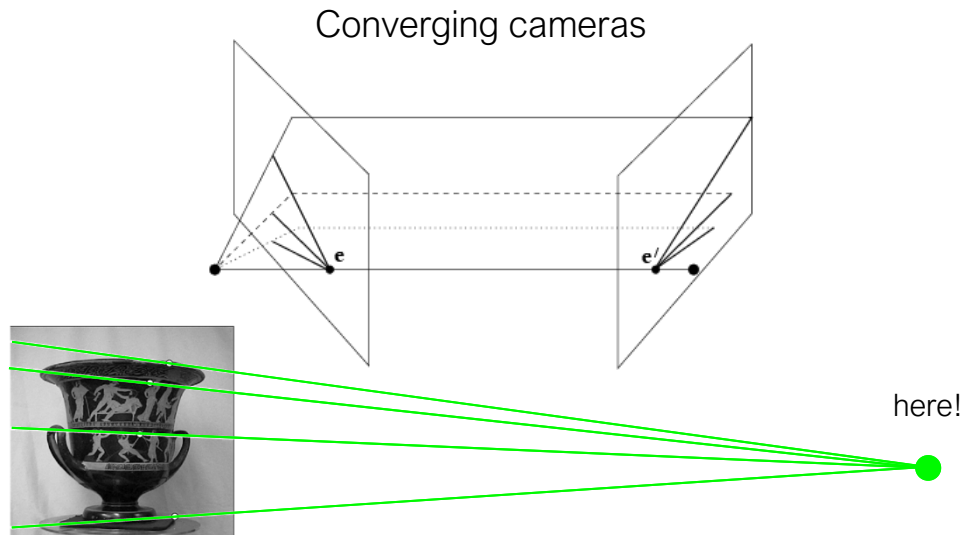
In particular note that e' is also on l' as it is a projection of C , and C is a 3D point on the ray connecting p and the camera center.

Converging cameras



Where is the epipole in this image?

Before we see how we can use this for 3D reconstruction, let's take a quick look at an example. We have two images of the same object by two cameras, visualized at the top. We can draw corresponding epipolar lines on the two images, as shown at the bottom. These epipolar lines give us a sense of how the two cameras are placed relative to each other.

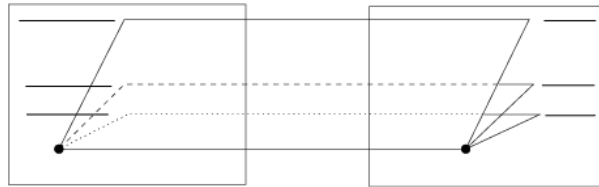


Where is the epipole in this image?

It's not always in the image

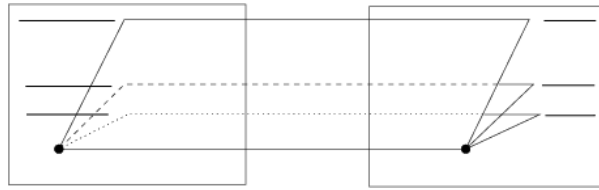
Note that the epipolar lines on an image intersect at some point. This point is the epipole we defined earlier. You can verify this by going back to the previous slides, and see how we constructed the epipolar planes and lines---you will see that by construction, all epipolar lines on an image pass through the epipole of that image. Note that, because the image is finite and does not cover the entire image plane, the epipole may actually be outside of the image.

Parallel cameras



Where is the epipole?

Parallel cameras



epipole at infinity

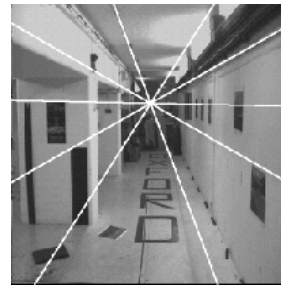
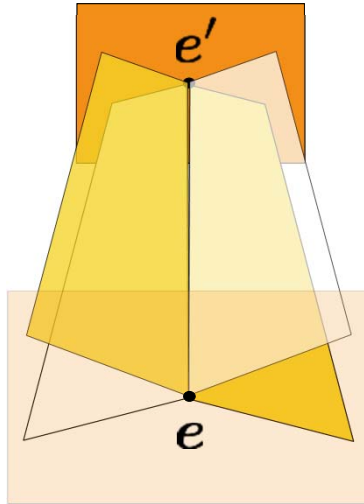
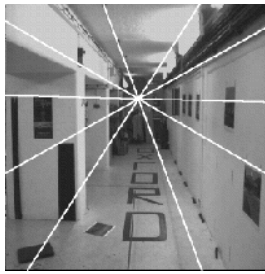
Forward moving camera



Forward moving camera



Epipole has same coordinates in both images.
Points move along lines radiating from “Focus of expansion”



The epipolar constraint is an important concept for stereo vision

Task: Match point in left image to point in right image



Left image

Right image

How would you do it?

Want to avoid search over entire image

Epipolar constraint reduces search to a single line

Next: How do you compute the epipolar line?

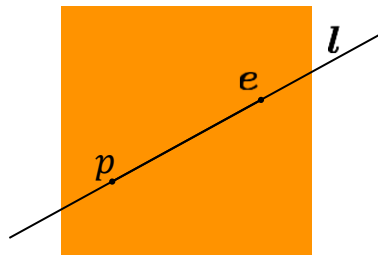
Alright, now let's go back to the matching task we had discussed in terms of 3D reconstruction. Given a point in the left image, we want to find the point in the right image that is looking at the same point in the 3D scene.

Previously, we thought that to do this matching, we would need to perform a 2D search on the right image. However, as we just saw, the epipolar constraint says that candidate matches are constrained to be on the right epipolar line corresponding to the image point we are trying to match. So, *if* we know the epipolar line, we can reduce our search for a match from a 2D search to an 1D search. In the next slides, we will see how we can compute the epipolar line.

Representing the ...

Epipolar Line

$$ax + by + c = 0 \quad \text{in vector form} \quad \mathbf{l} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$



If the point $\mathbf{p}$ is on the epipolar line $\mathbf{l}$ then

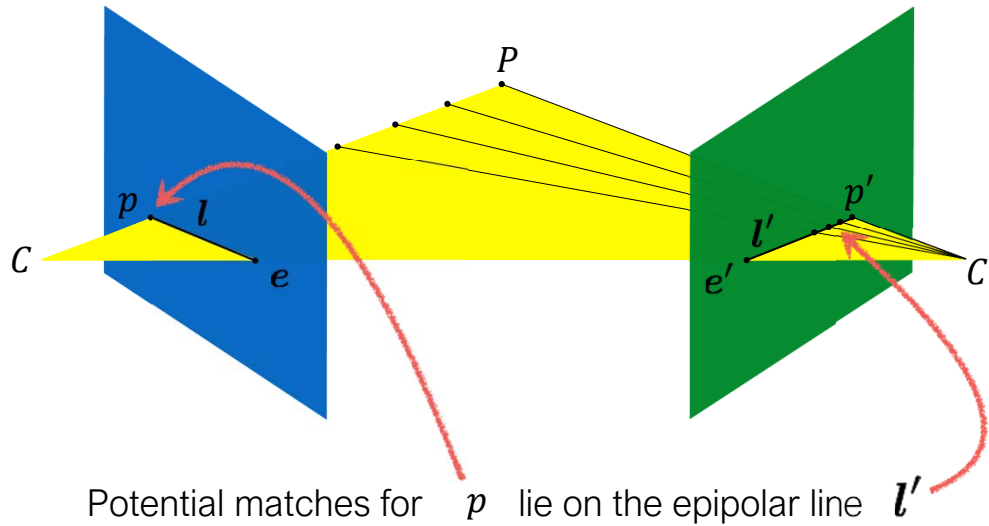
$$\mathbf{p}^T \mathbf{l} = 0$$

let's take a quick look at how we can express lines on a plane. From algebra, we know that we can write a line in the form $a \cdot x + b \cdot y + c = 0$, where x and y are the coordinates of points on the line, and a, b, c are parameters specifying the line.

What we can do then is stack the line parameters into a 3x1 vector $\mathbf{l}$. For any point in the 2D plane, if we express it in homogeneous coordinates as a 3x1 vector $\mathbf{p}$, then we can equivalently write the line equation as $\mathbf{p} \cdot \mathbf{l}$ equals zero. Therefore, knowing the 3x1 vector $\mathbf{l}$ is equivalent to knowing the line.

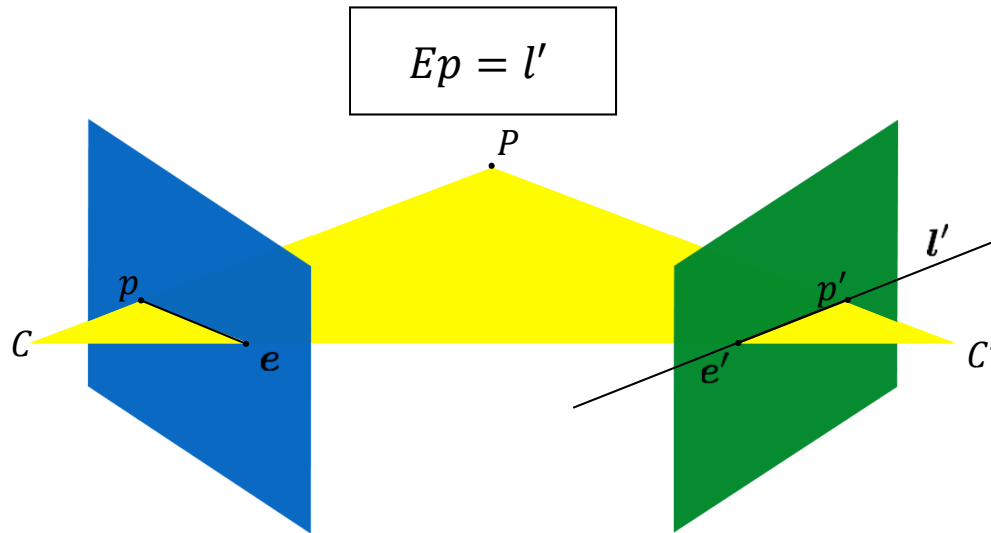
The essential matrix

Recall: Epipolar constraint



As a quick summary of what we covered so far, remember that epipolar geometry tells us that potential matches for a point p on the left image all line on the epipolar line l' on the right image.

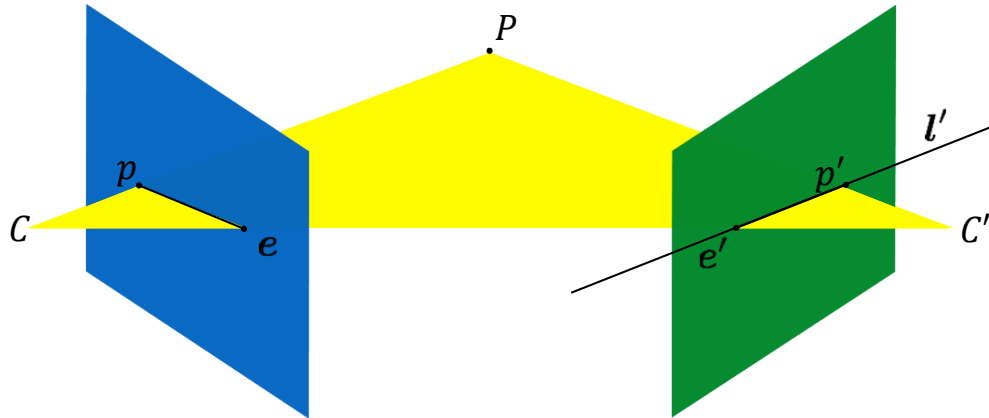
Given a point in one image,
multiplying by the **essential matrix** will tell us
the **epipolar line** in the second view.



What we will study in the coming slides is a special 3x3 matrix that we call the *essential matrix*, which describes this epipolar geometry. The essential matrix will have the following property: Given a point p on the left image, multiplying this point with the essential matrix E will give us a 3x1 vector corresponding to the epipolar line on the right image.

So if $p'^T l' = 0$ and $Ep = l'$ then

$$p'^T Ep = 0$$



we saw that multiplying the essential matrix E with the left image point p gives us the epipolar line l' . Now, let's say that we have a point on the right image p' that lies on this line. As we just described, mathematically this means that p' -times- l' equals zero. Putting this together gives us that left-and-right multiplying the essential matrix with two image points p and p' that satisfy the epipolar constraint gives us zero. If the result is not zero, then this means that p and p' do not satisfy the epipolar constraint, so p' cannot be the correspondence of p . This is a fundamental property of the essential matrix, and is usually the property we use to define it.

Essential Matrix vs Homography

What's the difference between the essential matrix and a homography?

They are both 3 x 3 matrices but ...

$$Ep = l'$$

Essential matrix maps a
point to a **line**

$$Hp = p'$$

Homography maps a
point to a **point**

When can we use a homography? And when only an essential matrix?

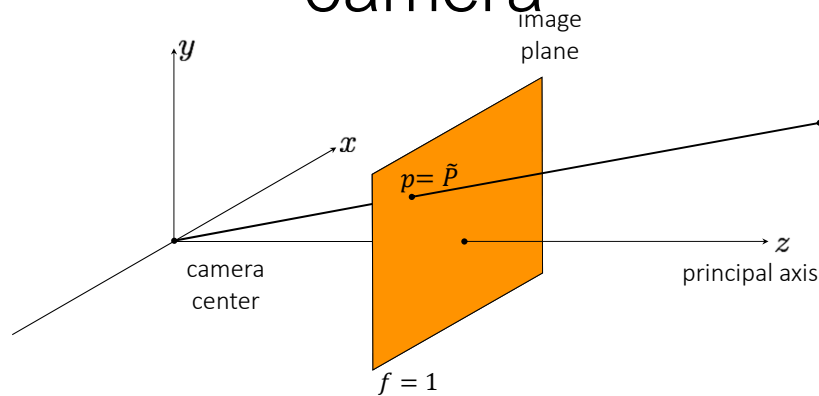
Homography applies only for planar scenes

Let's take a closer look at this essential matrix. First, note that it is a 3x3 matrix, same as the matrices we used to describe homographies. Even though the essential matrix and a homography are the same algebraically, their actual meaning is quite different: A homography maps a 2D point to another 2D point, whereas the essential matrix maps a 2D point to a line. Note that, all of these 2D points must be expressed using 3x1 homogeneous coordinate vectors.

Where does the essential matrix come from?
How can we express the essential matrix as
a function of geometric parameters?

So far, we have taken it for granted that the essential matrix exists. But where exactly does it come from? And how can we compute it? We will do a simple proof that the essential matrix exists, and also derive an expression for computing it from the geometric configuration of the two cameras.

The (rearranged) pinhole camera



3D heterogeneous coordinate $\tilde{p} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$, 2D homogeneous coordinate $p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$

Now recall that if we have a 2D image point p and we write it as a vector of 3D numbers $[x, y, 1]$ we essentially look at a 3D point located on the image plane. (assuming focal length $f=1$).

A note on coordinate frames

We assume the cameras are represented by the following matrices

$$\begin{aligned} \mathbf{M}_1 &= KR[I|0] & \mathbf{M}_2 &= KR[I] - dC \\ \text{or } \mathbf{M}_2 &= K [R|t] & \text{where } t &= -RdC \end{aligned}$$

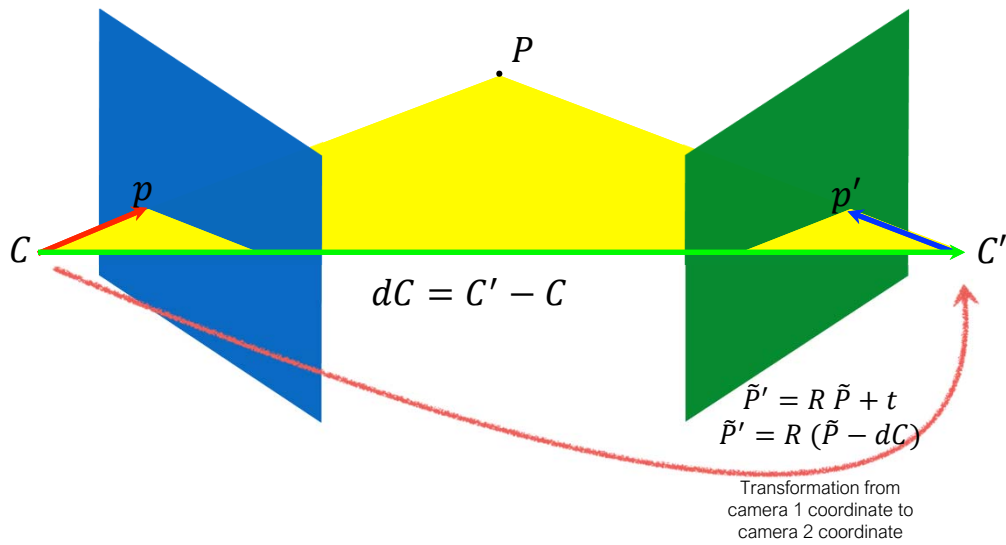
R, t are a transformation from the coordinate frame of camera 1 (which are also the world coordinates) to the coordinate frame of camera 2.

That is to transform a heterogeneous point $\tilde{P}$ from camera 1 coordinates to camera 2 coordinates we apply

$$R(\tilde{P} - dC) = R\tilde{P} + t$$

To perform the inverse mapping from coordinate frame of camera 2 to coordinate frame of camera 1 we do

$$\tilde{P} = R^T(\tilde{P}' - t) = R^T\tilde{P}' + R^T R dC = R^T\tilde{P}' + dC$$



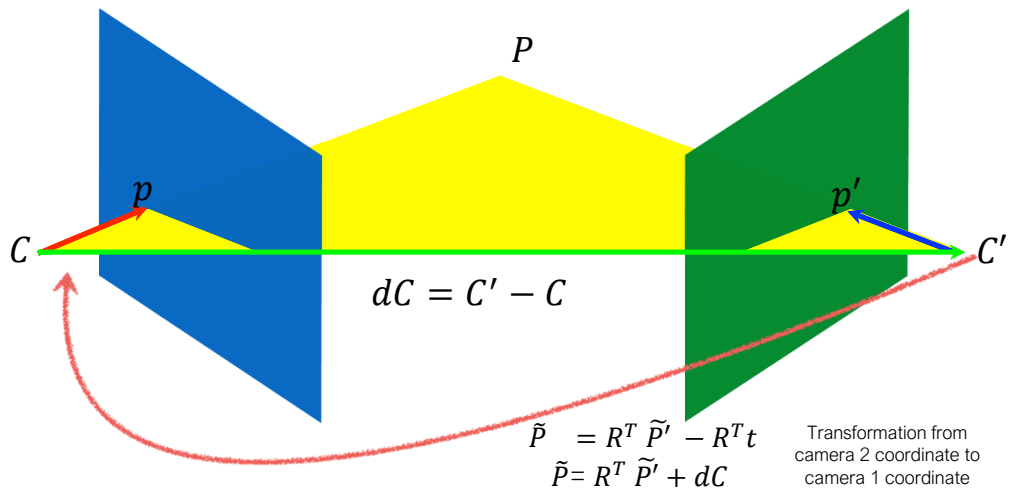
Camera-camera transform just like **world-camera** transform

In the coming set of slides, we will assume that 2D points on the cameras' image planes are expressed in the camera coordinate system. This is to say that the intrinsic matrices of the two cameras are identities.

Our goal is to derive an expression to the essential matrix.

For that, let's look again at our two cameras. We assume we know that we know the relative pose of the two cameras. In particular, we know the displacement vector dC describing how we can go from the center of camera 1 to the center of camera 2. And also, that we know a rotation matrix R that describes how we need to rotate the axes of the coordinate system of camera 2, to align with the axes of the coordinate system of camera 1.

In particular if we are given a 3D point in heterogeneous coordinates $\tilde{P}$ and this point is expressed at the coordinate frame of camera 2, we can transfer it to the coordinate frame of camera 1 using R t and dC .



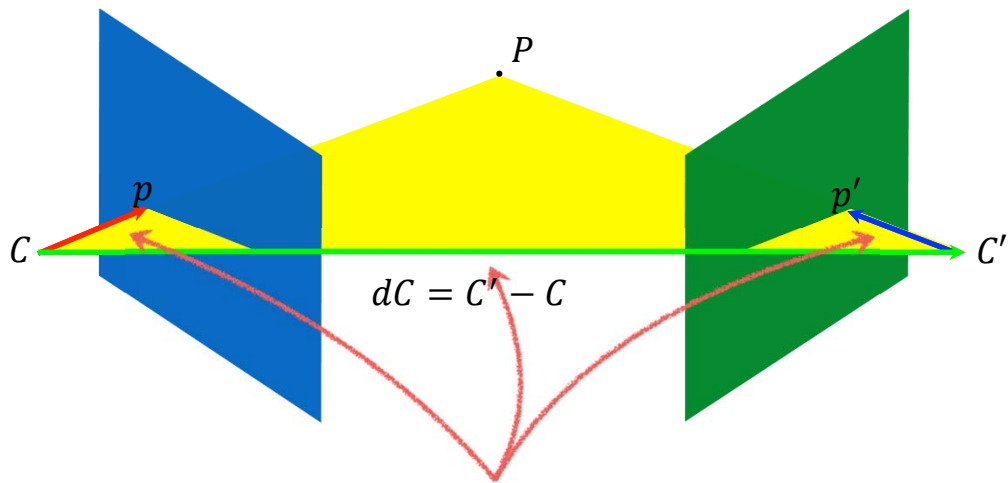
$$p'_{\text{coordinate frame 1}} = R^T p' - R^T t = R^T p' + dC$$

Camera-camera transform just like **world-camera** transform

So we can take the point p' in the second image and think of it as a 3D point $[x, y, 1]$

This is a point in the coordinate frame of the 2nd camera.

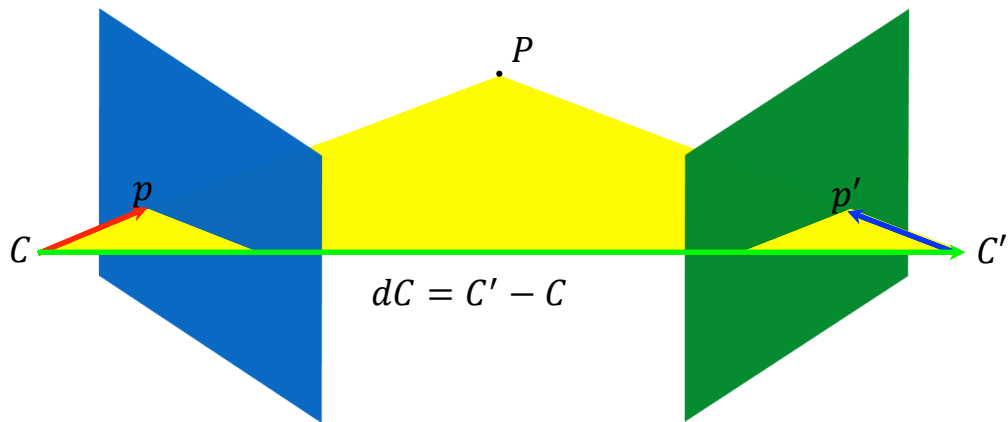
We can convert it and express it as a point in the coordinate frame of the 1st camera using R and t .



These three vectors are coplanar

p, dC, p' coordinate frame 1

Now, notice that the vectors describing the displacement vector dC and the points p and p' when both expressed in the coordinate systems of camera 1, are coplanar: They are all contained in the epipolar plane. In fact, dC is the same as the baseline vector we defined earlier.



If these three vectors are coplanar p, dC, p' coordinate frame 1

$$(p'_{\text{coordinate frame 1}})^T (dC \times p) = 0$$

We can use this coplanarity property to write down the equation we see here: We first compute the cross-product of dC and p , which gives us a vector orthogonal to the epipolar plane. We then take the dot product of this new vector with p' , which will give us zero, given that the result of the cross product is orthogonal to all vectors on the epipolar plane, including p' .

putting it together

coplanarity

$$(p'_{\text{coordinate frame 1}})^T (dC \times p) = 0$$

rigid motion

$$p'_{\text{coordinate frame 1}} = R^T p' + dC$$

$$(R^T p' + dC)^T (dC \times p) = 0$$

$$(p')^T R (dC \times p) = 0$$

So, let's now gather the equations we have written down.

We have an equation expressing the coplanarity of dC , p , and p' .

We also have an equation expressing the point in the left image in the coordinate frame of the 1st camera.

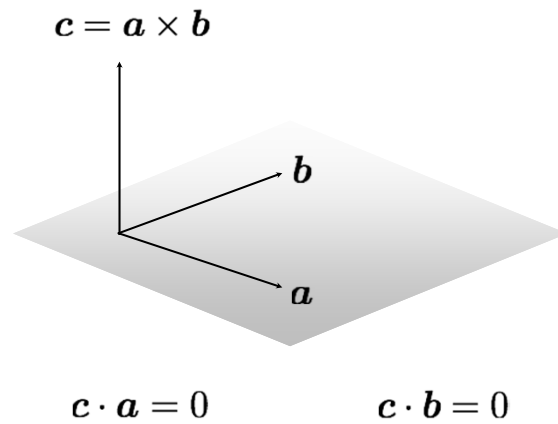
We can now combine these to arrive at an equation involving matrix vector products, dot products, and cross products between p , p' , R , and dC .

This equation will be the essential matrix but to express it we need to review some linear algebra.

Recall: Cross Product

Vector (cross) product

takes two vectors and returns a vector perpendicular to both



The cross product of 2 vectors a, b is a vector c orthogonal to the plane containing them.

Cross product

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$$

Can also be written as a matrix multiplication

$$\mathbf{a} \times \mathbf{b} = [\mathbf{a}]_{\times} \mathbf{b} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Skew symmetric

The cross product is given by this formula.

We can express cross products with a vector $\mathbf{a}$, as matrix-vector products with a special, skew-symmetric matrix formed from the elements of the vector $\mathbf{a}$.

putting it together

coplanarity

$$(p'_{\text{coordinate frame 1}})^T (dC \times p) = 0$$

rigid motion

$$p'_{\text{coordinate frame 1}} = R^T p' + dC$$

$$(R^T p' + dC)^T (dC \times p) = 0$$

$$(p')^T R (dC \times p) = 0$$

$$(p')^T (R[dC_{\times}])p = 0$$

$$(p')^T E p = 0$$

Essential Matrix
[Longuet-Higgins 1981]

Going back to our expression, we rewrite the cross product between dC and p as a matrix-vector product involving the skew-symmetric matrix of dC .

Now, we can rearrange terms to bring the rotation and skew symmetric matrices together.

At this point, we can recognize the product of R and the skew symmetric matrix $dC_{\times}$ as the essential matrix. Therefore, we have proven that the essential matrix exists, and can be expressed in terms of the rotation matrix R and translation vector dC explaining the relative poses of the two cameras. This result was proven in 1981 by Longuet-Higgins. For this and several other foundational contributions to computer vision, one of the most prestigious computer vision awards has been named after Longuet-Higgins.

The essential matrix

$$(p')^T E p = 0$$

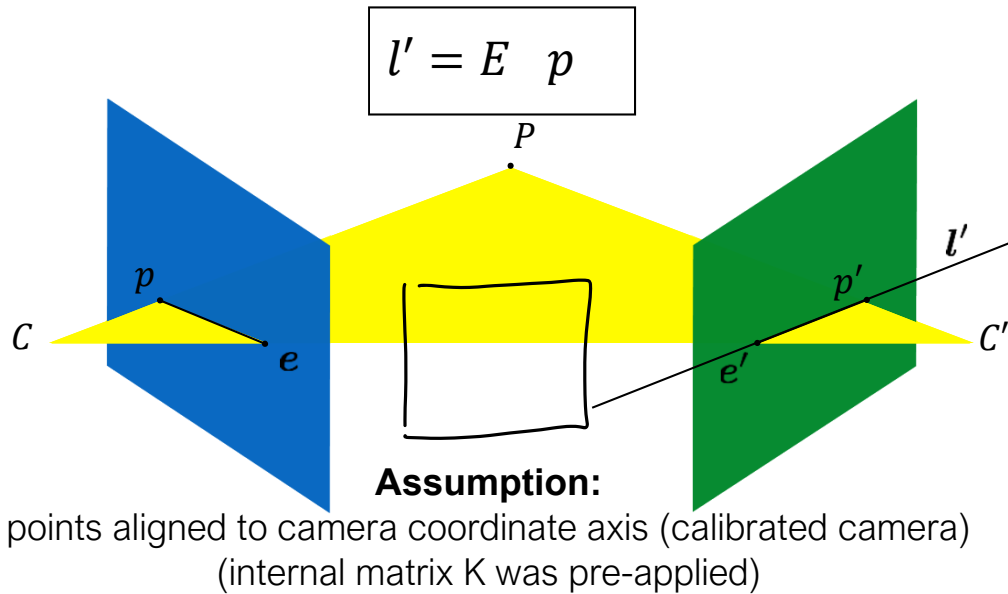
$$E = R[dC_{\times}]$$

Can also show $E = [t_{\times}]R$

with $t = -RdC$

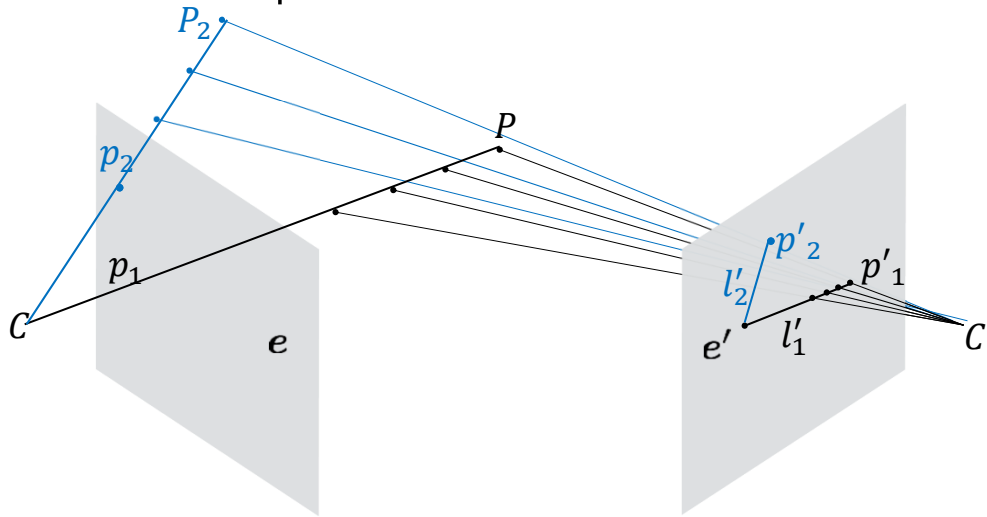
A more common way to express the essential matrix that you may encounter in the literature is shown here. It is a short exercise to show these are equivalent.

Given a point in one image,
multiplying by the **essential matrix** will tell us
the **epipolar line** in the second view.



Reminder: the essential matrix is mapping points in one image to epipolar lines in the second image.

The nullspace of the essential matrix



for every $p: e'^T l' = 0 \Rightarrow e'^T E p = 0 \Rightarrow e'^T E = 0$

We also so that all epipolar lines intersect at the epipole.

From that we can conclude that $e'^T E = 0$

כל הנקודות הנמצאות על הקו העpipolar יתנו את אותו התוצאה

properties of the E matrix

Longuet-Higgins equation

$$p'^T E p = 0$$

Epipolar lines

$$p^T l = 0$$

$$p'^T l' = 0$$

$$l' = E p$$

$$l = E^T p'$$

Epipoles

$$e'^T E = 0$$

$$E e = 0$$

Let's now summarize the properties of the essential matrix. Remember that, every time we are working with the essential matrix, we are expressing 2D points in homogeneous coordinates in the *camera* coordinate system, ignoring camera intrinsics. We have first the Longuet-Higgins equation, which says that left-and-right multiplying the essential matrix by two image points satisfying the epipolar constraint gives us zero.

Second, we have the property we started from: Given a point on one image, we can find the corresponding epipolar line on the other image through multiplication by the essential matrix.

Lastly, we have that when multiplying the epipoles with the essential matrix, we get zero. This mathematically expresses the fact that all epipolar lines pass through the two epipoles.

In all of this, we have been expressing image points in camera coordinates, which in practice only works if our cameras have intrinsic matrices that are identities. In the next slides, we will see how to remove this assumption.

How do you generalize to
uncalibrated cameras?

The fundamental matrix

The
Fundamental matrix
is a
generalization
of the
Essential matrix,
where the assumption of
calibrated cameras
is removed

we will see a different matrix that we will call the fundamental matrix. This is a generalization of the essential matrix that allows us to remove the assumption of identity intrinsic matrices, and express all image points on the image coordinate system.

$$\hat{p}'^T E \hat{p} = 0$$

The Essential matrix operates on image points expressed in
normalized coordinates
 (points have been aligned (normalized) to camera coordinates)

$$\hat{p}' = K^{-1} p' \quad \hat{p} = K^{-1} p$$

camera
image
point
point

We have how the essential matrix and the Longuet-Higgins equation can be used to check whether two image points, expressed in the camera coordinate system, satisfy the epipolar constraint. As we saw in the Camera Models lecture, for each of the two points, we can relate image coordinates and camera coordinates using the corresponding camera intrinsic matrices K and K' .

$$\hat{p}'^T E \hat{p} = 0$$

The Essential matrix operates on image points expressed in **normalized coordinates**
(points have been aligned (normalized) to camera coordinates)

$$\hat{p}' = K'^{-1} p' \quad \hat{p} = K^{-1} p$$

camera point image point

Writing out the epipolar constraint in terms of image coordinates

$$p'^T K'^{-T} E K^{-1} p = 0$$

$$p'^T (K'^{-T} E K^{-1}) p = 0$$

$$p'^T F p = 0$$

We can then replace the camera coordinates with image coordinates in the Longuet-Higgins equation, and define a new matrix F that combines the essential matrix and the intrinsic matrices.

Same equation works in image coordinates!

$$p'^T F p = 0$$

it maps pixels to epipolar lines

With this simple procedure, we arrive at a new equation that is identical to the Longuet-Higgins equation, but is now expressed in image coordinates. We call this new matrix F the fundamental matrix.

properties of the ~~E~~ matrix

Longuet-Higgins equation

$$p'^T \mathbf{F} p = 0$$

Epipolar lines

$$p^T l = 0$$

$$p'^T l' = 0$$

$$l' = \mathbf{F} p$$

$$l = \mathbf{F}^T p'$$

Epipoles

$$e'^T \mathbf{F} = 0$$

$$\mathbf{F} e = 0$$

The fundamental matrix has all the same properties as the essential matrix, except now with 2D points expressed in the image coordinate system. These include the Longuet-Higgins equation, the mapping from image points to epipolar lines, and the definition of the epipoles.

Breaking down the fundamental matrix

$$\mathbf{F} = \mathbf{K}'^{-\top} \mathbf{E} \mathbf{K}^{-1}$$

$$\mathbf{F} = \mathbf{K}'^{-\top} [\mathbf{t}_\times] \mathbf{R} \mathbf{K}^{-1}$$

Depends on both intrinsic and extrinsic parameters

We had previously expressed the essential matrix in terms of the camera extrinsics, namely the rotation vector $\mathbf{R}$ and translation vector $\mathbf{t}$ relating their coordinate systems. Given how we defined the fundamental matrix, we can use those, along with the camera intrinsic matrices, to express the fundamental matrix. This expression says that, if we have a fully calibrated pair of cameras, we can use their intrinsic and extrinsic matrices to compute their fundamental matrix. We will see in the next slides that calibrating the two cameras is not necessary for computing their fundamental matrix. Instead, we can compute it using a set of matched point pairs between two images captured by the two cameras.

Differences between a fundamental matrix and an homography

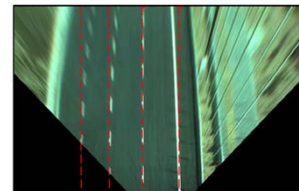
Homography: maps point to point
Fundamental matrix only maps points to lines

Homography applies only to planes
Fundamental matrix explains the entire 3D scene

Caution: If your scene is planar or the cameras have a joint center, it can be explained by an homography but this homography is *not* the Fundamental matrix



Original Road Image



Breaking down the fundamental matrix

$$\mathbf{F} = \mathbf{K}'^{-\top} \mathbf{E} \mathbf{K}^{-1}$$

$$\mathbf{F} = \mathbf{K}'^{-\top} [\mathbf{t}_\times] \mathbf{R} \mathbf{K}^{-1}$$

Depends on both intrinsic and extrinsic parameters

How would you solve for F ?

$$p_m'^T F p_m = 0$$

The 8-point algorithm

Assume you have M matched *image* points

$$\{ p_j, p'_j \}$$

Each correspondence should satisfy

$$p_j'^T F p_j = 0$$

How would you solve for the 3 x 3 F matrix?

Set up a homogeneous linear system with 9 unknowns

$$S \quad V \quad D$$

In the previous slides, we saw how we can compute the fundamental matrix using the intrinsics and extrinsics of two calibrated cameras. In this submodule, we will see how to do the same using a set of matched image points. In particular, we assume we have a set of N pairs of points on the left and right images that are matched, meaning that they look at the same 3D point in the scene. Given this, each image point pair satisfies the Longuet-Higgins equation. Our task will be to solve for the 3x3 fundamental matrix F . We will use the same high-level procedure we have been using for all these reconstruction tasks: we will try to express our constraints in the form of a linear system that we can then solve for the unknown fundamental matrix F .

$$p_m'^T F p_m = 0$$

$$\begin{bmatrix} x'_m & y'_m & 1 \end{bmatrix} \begin{bmatrix} f_1 & f_2 & f_3 \\ f_4 & f_5 & f_6 \\ f_7 & f_8 & f_9 \end{bmatrix} \begin{bmatrix} x_m \\ y_m \\ 1 \end{bmatrix} = 0$$

How many equation do you get from one correspondence?

$$\begin{bmatrix} x'_m & y'_m & 1 \end{bmatrix} \begin{bmatrix} f_1 & f_2 & f_3 \\ f_4 & f_5 & f_6 \\ f_7 & f_8 & f_9 \end{bmatrix} \begin{bmatrix} x_m \\ y_m \\ 1 \end{bmatrix} = 0$$

Set up a homogeneous linear system with 9 unknowns

$$\begin{bmatrix} x_1x'_1 & x_1y'_1 & x_1 & y_1x'_1 & y_1y'_1 & y_1 & x'_1 & y'_1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_Mx'_M & x_My'_M & x_M & y_Mx'_M & y_My'_M & y_M & x'_M & y'_M & 1 \end{bmatrix} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \\ f_8 \\ f_9 \end{bmatrix} = \mathbf{0}$$

How many equations do you need?

We start by writing out the Longuet-Higgins equation for one image pair.

we can expand out the matrix vector multiplications to express this as a long linear equation involving point coordinates (entries of the p, p' points) and matrix entries. We reshape the 3x3 elements of F into an unknown 9x1 vector.

Stacking equations from all point pairs gives us a homogeneous linear system, where the unknowns are the 9 entries of the 3x3 fundamental matrix.

As always, we can solve this homogeneous linear system using the SVD.

Looking at the system, we see that we have 8 degrees of freedom. The system has 9 unknowns, but remember that fundamental matrices use homogeneous coordinates, and therefore they are defined up to scale, which removes one degree of freedom.

Each point pair (according to epipolar constraint)
contributes only one scalar equation

$$p_m'^T F p_m = 0$$

Note: This is different from the Homography estimation where
each point pair contributes 2 equations.

We need at least 8 points

Hence, the 8 point algorithm!

We also see that every point pair provides us with one constraint on the
unknowns.

Therefore, we need at least 8 point pairs to uniquely determine the fundamental
matrix, which is where the name 8-point algorithm comes from.

How do you solve a homogeneous linear system?

$$\mathbf{A}\mathbf{X} = \mathbf{0}$$

Total Least Squares

minimize $\|\mathbf{A}\mathbf{x}\|^2$

subject to $\|\mathbf{x}\|^2 = 1$

S V D !

Remember also, from previous lectures, that when we solve homogeneous linear systems, we add a unit-norm constraint. That constraint corresponds to the one degree of freedom we lose due to scale invariance.

As before we solve this with SVD.

Eight-Point Algorithm

0. (Normalize points)

1. Construct the $M \times 9$ matrix $\mathbf{A}$
2. Find the SVD of $\mathbf{A}$
3. Entries of $\mathbf{F}$ are the elements of column of $\mathbf{V}$ corresponding to the least singular value

4. (Enforce rank 2 constraint on $\mathbf{F}$)

5. (Un-normalize $\mathbf{F}$)

Here is a summary of the eight point algorithm. Note that, in green, there are a few steps that we have not described. We will discuss the normalization and unnormalization steps in the next slides.

Eight-Point Algorithm

0. (Normalize points)

1. Construct the $M \times 9$ matrix $\mathbf{A}$
2. Find the SVD of $\mathbf{A}$ Here $\mathbf{F}$ is a 9×1 vector
3. Entries of $\mathbf{F}$ are the elements of column of $\mathbf{V}$ corresponding to the least singular value

4. (Enforce rank 2 constraint on $\mathbf{F}$)

5. (Un-normalize $\mathbf{F}$)

This is a 2nd SVD on $\mathbf{F}$ itself, now rearranged as a 3×3 matrix



The fundamental matrix should be a rank 2 matrix. This results from the fact that $\mathbf{F}\mathbf{e}=0$.

However when we solved for $\mathbf{F}$ we expressed it as a vector of 9×1 numbers. When we solve this system we get a 9 elements vector out, and we reshape them into a 3×3 matrix. This matrix is unlikely to be a rank 2 matrix because it is a non linear constraint on the elements of $\mathbf{F}$, that we did not expressed in the formation of the constraints in $\mathbf{A}$.

As we described in the Homographies lecture, we can enforce the rank 2 constraint by computing the singular value decomposition of $\mathbf{F}$, and setting its smallest singular value to zero.

Note that we use here SVD twice. First SVD on the big matrix $\mathbf{A}$ to get the 9 elements of $\mathbf{F}$.

We then reshape them into a 3×3 matrix and apply SVD on this 3×3 matrix to make it a rank 2 matrix

Reminder from lecture 6: Using SVD to enforce rank constraints

Problem statement: Given

- an $M \times N$ matrix F ,
- an integer $k < \min(M, N)$,

find the matrix F' of rank k that is closest to F in the least-squares sense

$$\min_{\substack{F' \\ \text{rank}(F')=k}} \|F - F'\|^2$$

Solution:

- compute the SVD $F = U \cdot \Sigma \cdot V^T$
- form a diagonal matrix Σ' by replacing all but the k smallest singular values in Σ with 0.
- Compute the solution as $F' = U \cdot \Sigma' \cdot V^T$

Before we finish this linear algebra detour, I want to mention one more property of SVD we will be using in later lectures. Consider the following problem: We have an M by N matrix F , and we want to approximate it, in the least squares sense, with a new matrix F' , of some lower rank k .

We can use the SVD of F to compute the new matrix F' as follows: Take the singular value matrix Σ , set all singular values except for the k largest to zero, then form F' by left-and-right multiplying the resulting matrix with U and V -transpose.

Eight-Point Algorithm

Note: in practice F has only 7 degrees of freedom (why?),

0. (Normalize points)

1. Construct the $M \times 9$ matrix $\mathbf{A}$
2. Find the SVD of $\mathbf{A}$
3. Entries of $\mathbf{F}$ are the elements of column of $\mathbf{V}$ corresponding to the least singular value

4. (Enforce rank 2 constraint on F)

5. (Un-normalize F)

Note that in practice this implies that the Fundamental matrix has only 7 degrees of freedom not 8.

The fact that F needs to be a rank 1 matrix adds an additional constraint.

However this is a non linear constraint and we do not use it as part of our linear system when forming the matrix A . Therefore to solve for F using a linear system we need at least 8 points.

If we have only 7 points we will need to include non linear constraints.

Eight-Point Algorithm

Note: in practice F has only 7 degrees of freedom (rank 2), but the last constraint is non linear and is not used by the 8 point algorithm

0. (Normalize points)
1. Construct the $M \times 9$ matrix $\mathbf{A}$
2. Find the SVD of $\mathbf{A}$
3. Entries of $\mathbf{F}$ are the elements of column of $\mathbf{V}$ corresponding to the least singular value
4. (Enforce rank 2 constraint on F)
5. (Un-normalize F)

Example



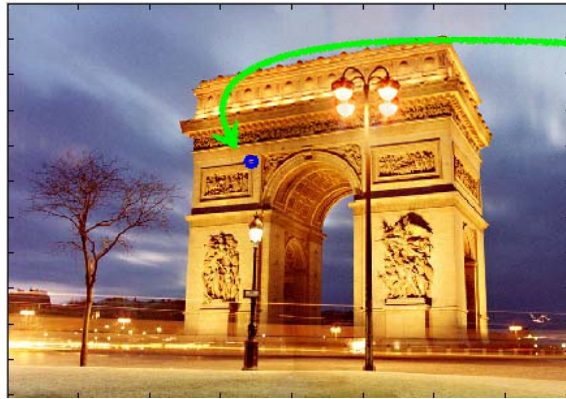
Let's look at a quick example of how this all works out. We are given these two images of the same scene, the arc de triomphe in Paris. We do not have any information about the cameras that captured these two images, we just know that they are looking at the same scene.

epipolar lines



We can use the techniques from the Feature Detection and Matching lecture to find at least 8 matching point pairs between the two images. We can then use those as input to our implementation of the 8-point algorithm, to compute the fundamental matrix relating the two images. Using the fundamental matrix, we can, for example, draw corresponding epipolar lines on the two images.

$$\mathbf{F} = \begin{bmatrix} -0.00310695 & -0.0025646 & 2.96584 \\ -0.028094 & -0.00771621 & 56.3813 \\ 13.1905 & -29.2007 & -9999.79 \end{bmatrix}$$



$$p = \begin{bmatrix} 343.53 \\ 221.70 \\ 1.0 \end{bmatrix}$$

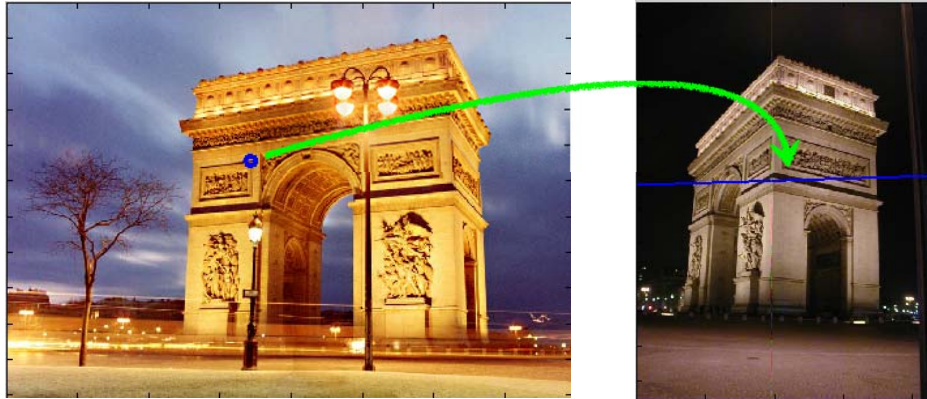
$$\begin{aligned} l' &= Fp \\ &= \begin{bmatrix} 0.0295 \\ 0.9996 \\ -265.1531 \end{bmatrix} \end{aligned}$$

With the fundamental matrix at hand, we can also take some arbitrary point on image one...

... and compute and draw the corresponding epipolar line on image two. Note that the epipolar line indeed passes through the point corresponding to the corner we selected in image one.

$$l' = Fp$$

$$= \begin{bmatrix} 0.0295 \\ 0.9996 \\ -265.1531 \end{bmatrix}$$



With the fundamental matrix at hand, we can also take some arbitrary point on image one...

... and compute and draw the corresponding epipolar line on image two. Note that the epipolar line indeed passes through the point corresponding to the corner we selected in image one.

Where is the epipole?



How would you compute it?

Lastly, remember that the epipolar lines we drew intersect at the epipole.
The epipole may lay outside the image boundaries.



$$\mathbf{F} \mathbf{e} = \mathbf{0}$$

The epipole is in the right null space of $\mathbf{F}$

How would you solve for the epipole?

(hint: this is a homogeneous linear system)

S V D !

As we saw earlier, the epipole when multiplied by the fundamental matrix $\mathbf{F}$ gives us zero. Therefore, we can compute the epipole by solving a homogeneous linear system involving the fundamental matrix.



```
>> [u,d] = eigs(F' * F)
eigenvectors
u =
   -0.0013    0.2586   -0.9660
    0.0029   -0.9660   -0.2586
    1.0000    0.0032   -0.0005

eigenvalue
d = 1.0e8*
    1.0000         0         0
         0    0.1000         0
         0         0    0.0000
```

this means that we can find the epipole by simply computing the SVD of F , and taking the right singular vector corresponding to the smallest singular value.



```
>> [u,d] = eigs(F' * F)
```

eigenvectors

u =

-0.0013	0.2586	-0.9660
0.0029	-0.9660	-0.2586
1.0000	0.0032	-0.0005

eigenvalue

d = 1.0e8*

1.0000	0	0
0	0.1000	0
0	0	0.0000



```
>> [u,d] = eigs(F' * F)
```

eigenvectors

u =

-0.0013	0.2586	-0.9660
0.0029	-0.9660	-0.2586
1.0000	0.0032	-0.0005

eigenvalue

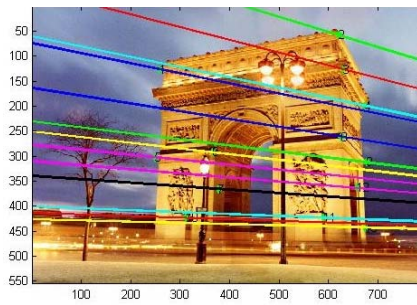
d = 1.0e8*

1.0000	0	0
0	0.1000	0
0	0	0.0000

Eigenvector associated with
smallest eigenvalue

```
>> uu = u(:,3)
```

```
( -0.9660 -0.2586 -0.0005)
```



```
>> [u,d] = eigs(F' * F)
```

```
eigenvectors
```

```
u =
```

-0.0013	0.2586	-0.9660
0.0029	-0.9660	-0.2586
1.0000	0.0032	-0.0005

```
eigenvalue
```

```
d = 1.0e8*
```

-1.0000	0	0
0	-0.0000	0
0	0	-0.0000

Eigenvector associated with
smallest eigenvalue

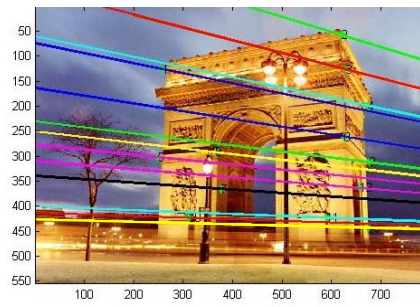
```
>> uu = u(:,3)
```

```
( -0.9660    -0.2586    -0.0005)
```

Epipole projected to image
coordinates

```
>> uu / uu(3)
```

```
(1861.02    498.21    1.0)
```



epipole

Epipole projected to image
coordinates

```
>> uu / uu(3)
(1861.02    498.21    1.0)
```

The Fundamental Matrix Song



[https://www.youtube.com/
watch?v=DgGV3l82NTk](https://www.youtube.com/watch?v=DgGV3l82NTk)

A note on normalization

Estimating F – 8-point algorithm

- The fundamental matrix F is defined by

$$p_m'^T F p_m = 0$$

for any pair of matches p and p' in two images.

- Let $x=(u,v,1)^T$ and $x'=(u',v',1)^T$,
$$\mathbf{F} = \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix}$$
 each match gives a linear equation

$$uu' f_{11} + vu' f_{12} + u' f_{13} + uv' f_{21} + vv' f_{22} + v' f_{23} + uf_{31} + vf_{32} + f_{33} = 0$$

We will now explain why we need to use normalization to make the 8-point algorithm work robustly. For this, we can take a look at the homogeneous linear system we are solving to compute the fundamental matrix F.

Problem with 8-point algorithm

$$\begin{bmatrix}
 u_1 u_1' & v_1 u_1' & u_1' & u_1 v_1' & v_1 v_1' & v_1' & u_1 & v_1 & 1 \\
 u_2 u_2' & v_2 u_2' & u_2' & u_2 v_2' & v_2 v_2' & v_2' & u_2 & v_2 & 1 \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 u_n u_n' & v_n u_n' & u_n' & u_n v_n' & v_n v_n' & v_n' & u_n & v_n & 1
 \end{bmatrix}
 \begin{bmatrix}
 f_{11} \\
 f_{12} \\
 f_{13} \\
 f_{21} \\
 f_{22} \\
 f_{23} \\
 f_{31} \\
 f_{32} \\
 f_{33}
 \end{bmatrix}
 = 0$$

$\sim 10000 \quad \sim 10000 \quad \sim 100 \quad \sim 10000 \quad \sim 10000 \quad \sim 100 \quad \sim 100 \quad \sim 100 \quad 1$

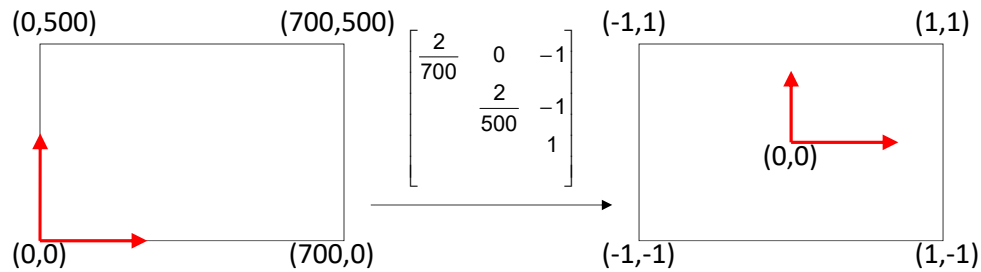
Orders of magnitude difference
between column of data matrix
→ least-squares yields poor results

Note that some of the columns involve products of two image coordinates, some involve individual image coordinates, and one column is equal to just ones. If our images have, say, 1024-by-1024 pixels, this means that the columns involving products of image coordinates will have entries magnitude around 10,000; and the columns involving individual image coordinates will have entries of magnitude around 100. We see that there are columns spanning four different orders of magnitude in the matrix of our linear system. This is problematic, because our marking of the points p, p' in the images will have noise. least-squares procedures are not robust in such situations: the results tend to overemphasize small errors in the large-magnitude columns, and completely ignore the small-magnitude ones.

Normalized 8-point algorithm

normalized least squares yields good results

Transform image to $\sim[-1,1] \times [-1,1]$



Fortunately, there is a very simple way to deal with this issue. Before running the 8-point algorithm, we can apply a similarity transform to each image, to map it to the $[-1,1] \times [-1,1]$ cube. As a consequence, the coordinates of the image points we use as input to the 8-point algorithm will now all have magnitudes between $[0, 1]$, rather than between $[0, 1000]$.

Normalized 8-point algorithm

1. Transform input by $\hat{p}_i = T p_i$, $\hat{p}'_i = T p'_i$
2. Call 8-point on $\hat{p}_i, \hat{p}'_i$ to obtain $\hat{F}$
3. $F = T'^T \hat{F} T$

$$p'^T F p = 0$$

$$\boxed{\hat{p}^T T'^{-T} F T^{-1} \hat{p} = 0}$$

$\underbrace{\hspace{10em}}_{\hat{F}}$

So, here is how this works. First, we transform the input points using the similarity transform we just described. Second, we run the 8-point algorithm on the transformed image-point pairs, to compute an intermediate fundamental matrix $\hat{F}$. This intermediate matrix is not what we want, given that it applies to points in the transformed image plane, and not the original image plane. To deal with this, our last step is to transform the intermediate to the final fundamental matrix by using multiplications with the similarity transforms.

To see where the expression mapping the intermediate to the final fundamental matrix comes from, consider first the Longuet-Higgins equation between the final fundamental matrix and the unnormalized image points.

We can replace in this equation the unnormalized image points with the normalized ones, using the inverse of the similarity transforms.

From this expression, we recognize the three inner terms as being equal to the intermediate fundamental matrix that satisfies the Longuet-Higgins equation for the normalized image points. We can then solve this equality for the final fundamental matrix, to obtain the expression we used in our third step.

Normalized 8-point algorithm

```
[p1, T1] = normalise2dpts(p1);
[p2, T2] = normalise2dpts(p2);

A = [p2(1,:)'.*p1(1,:) '    p2(1,:)'.*p1(2,:) '    p2(1,:) ' ...
     p2(2,:)'.*p1(1,:) '    p2(2,:)'.*p1(2,:) '    p2(2,:) ' ...
     p1(1,:) '              p1(2,:) '              ones(npts,1) ];

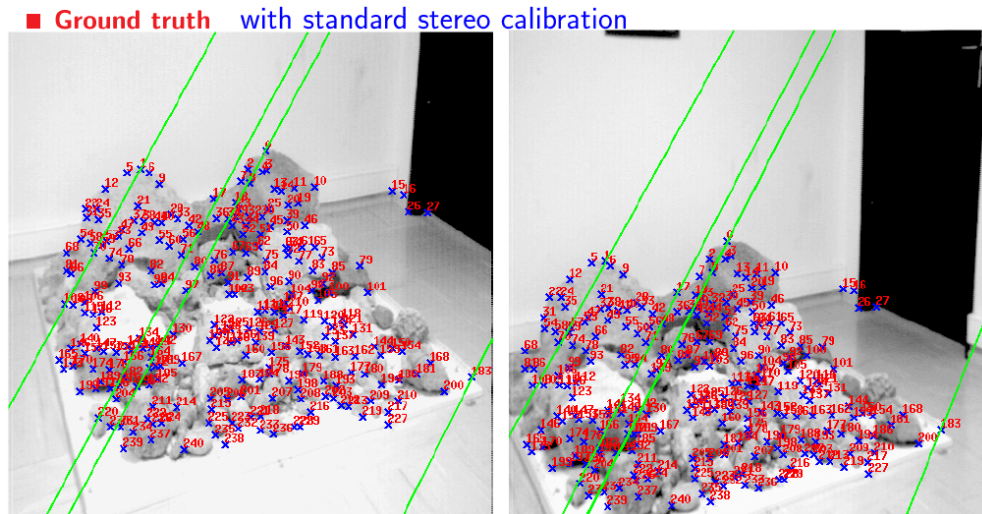
[U,D,V] = svd(A);

F = reshape(V(:,9),3,3)';

[U,D,V] = svd(F);
F = U*diag([D(1,1) D(2,2) 0])*V';

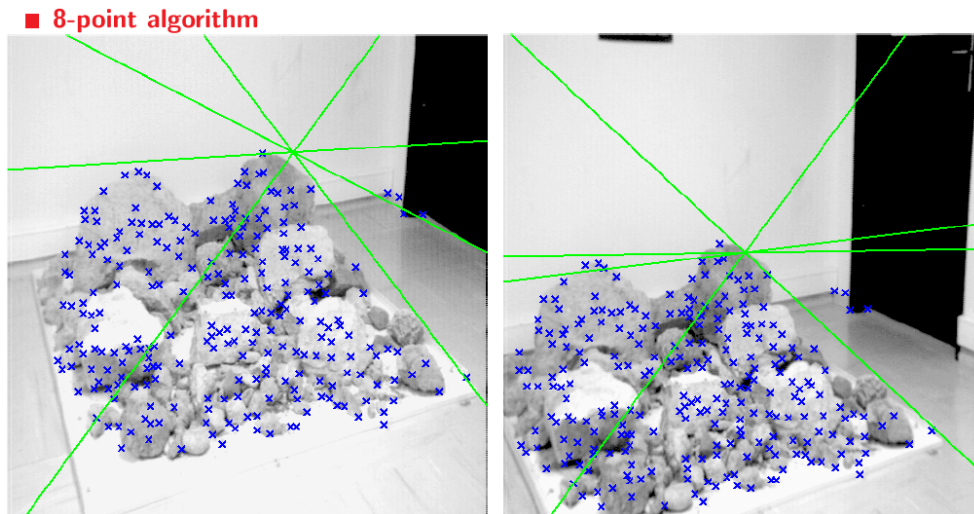
% Denormalise
F = T2'*F*T1;
```

Results (ground truth)



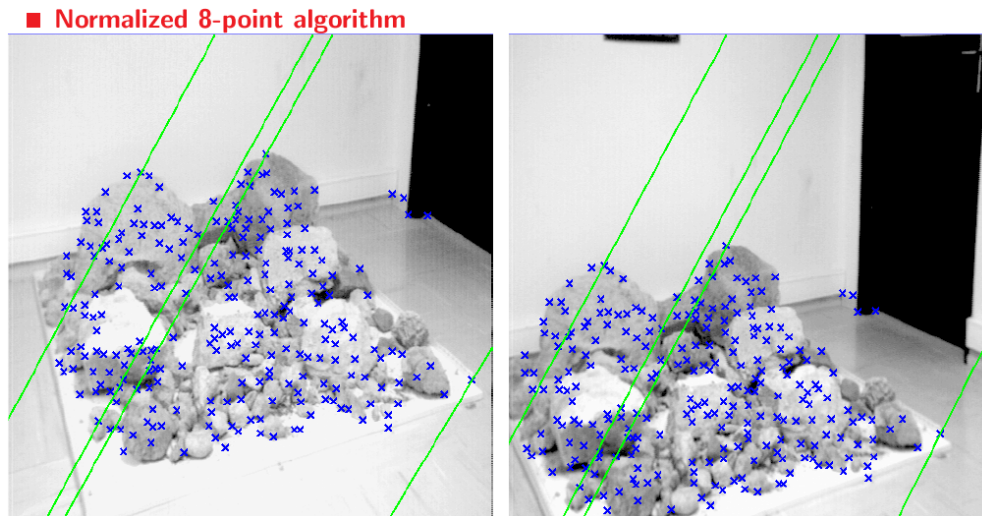
Here is a quick example of the impact normalization can have. We have two images, where we draw groundtruth epipolar lines, computed using standard camera calibration.

Results (8-point algorithm)



If we run the 8-point algorithm without normalization, and use the fundamental matrix we obtain to draw epipolar lines, we see that the results are completely different from the groundtruth. What has happened is that the linear-least-squares solution has overfitted to the large-magnitude entries in our matrix, producing incorrect epipoles near the centers of the images.

Results (normalized 8-point algorithm)



Note: Normalization procedure can benefit other geometry-based vision routines (e.g., DLT for homography estimation, camera calibration)

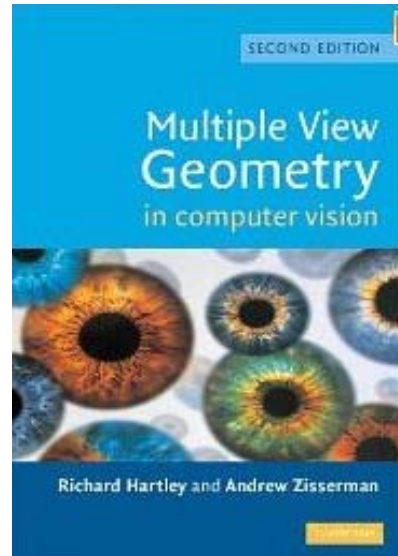
By contrast, if we run the 8-point algorithm using normalization, and then use the resulting fundamental matrix to draw epipolar lines, the results we get are near-identical to the groundtruth. We see, then, how big of a difference using normalization makes.

We have described the normalization procedure in the context of the 8-point algorithm. But note that normalization can benefit all of the geometry-based vision algorithms we have seen in this and the previous two modules, including the direct linear transform, camera calibration, and so on. All of those algorithms have variants that use normalization and unnormalization steps at the start and end.

References

Basic reading:

- Szeliski textbook, Sections 7.1, 7.2, 11.1.
- Hartley and Zisserman, Chapters 9, 11, 12.



Ch